

InclusiveCity

Documento di Analisi e

Progettazione

Corso di Ingegneria del Software

Nasuelli Chiara

Martinelli Alessandro

Raineri Luca

Introduzione, analisi approfondita del progetto

Obbiettivo del progetto

Il progetto è un'applicazione web progressiva che permette un routing dedicato in base alle disabilità di uno specifico individuo.

Il cuore dell'applicazione è una mappa che mostra una serie di barriere architettoniche, che vengono segnalate dagli utenti dell'applicazione. Un utente può anche dare valutazioni alla barriera inserita da un altro utente.

Deve essere presente anche un sistema di moderazione, in parte manuale (con degli utenti admin) e in parte automatica, sfruttando le valutazioni degli utenti.

Glossario

- **disabilità:** è una caratteristica di un utente. Può essere di vari tipi: stampelle, sedia a rotelle. Un utente può anche non avere disabilità, ma semplicemente voler contribuire inserendo segnalazioni di barriere.
- **barriera:** è un ostacolo presente in città. Può essere di vari tipi, ad esempio gradino, scale, marciapiede, buca, etc. Ha una posizione, un limite di superamento basato sulla disabilità dell'utente corrente. Una barriera può essere attiva o risolta, in revisione o nascosta. Sulla mappa vengono mostrate solamente le barriere attive.
- **reputation score:** livello di affidabilità di un utente.
- **feedback:** voto da 1 a 5 stelle alla correttezza della singola barriera. Può avere un commento testuale.
- **report:** segnalazione di inappropriatezza di una barriera. Ha un motivo (esempio: duplicato, posizione errata, ...).
- **resolution:** segnalazione di risoluzione di una barriera.

Metodologia di lavoro Agile applicata

La metodologia utilizzata è stata una metodologia Agile, incentrata su iterazioni con obiettivi precisi e produzione del codice a micro-incrementi incrementali ed iterativi. Questo ha permesso un controllo costante sul lavoro svolto e sulle funzioni mancanti, oltre a permettere una certa agilità nello sviluppo e nella correzione del lavoro da svolgere anche in fasi critiche.

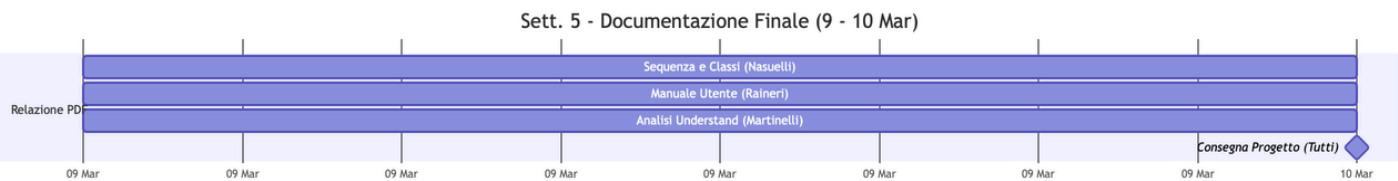
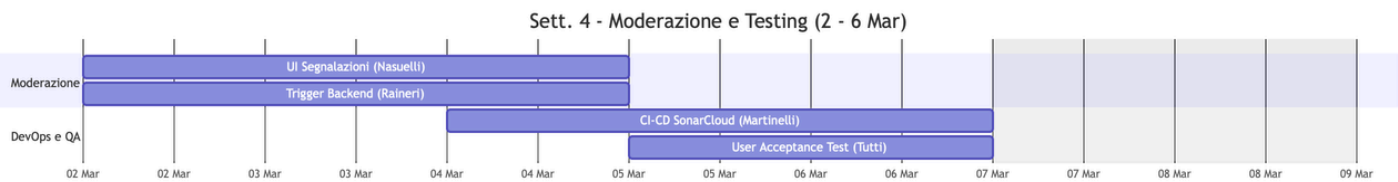
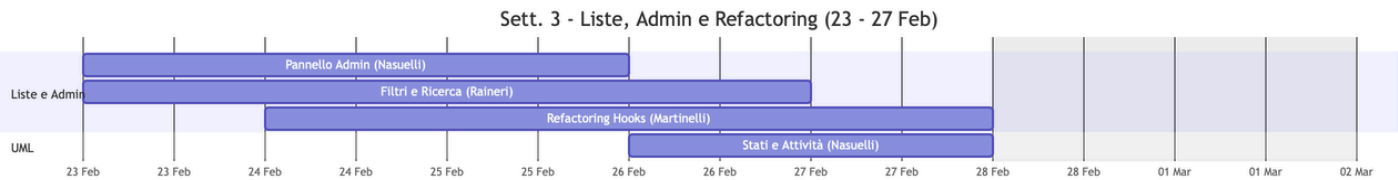
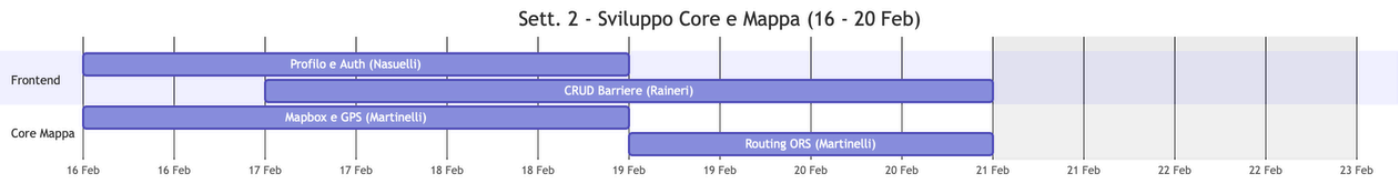
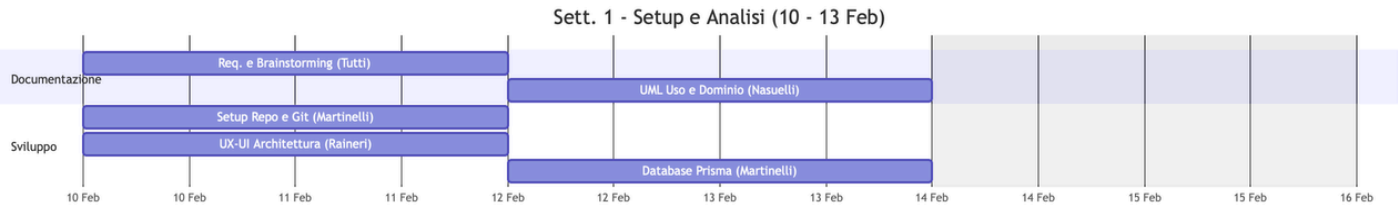
Le pratiche adottate sono state quelle di incontri di allineamento ogni 2-3 giorni, suddivisione per aree dei compiti e sessioni di pair-programming per eliminare problematiche complesse.

Le fasi principali sono state 3:

- **Iterazione 0 (definizione obiettivo e inizio analisi):** tutto il gruppo, lavorando assieme, ha definito con più precisione l'obiettivo del progetto. Durante questa fase, della durata di circa 3 giorni, sono stati definiti il primo diagramma dei casi d'uso e un primo diagramma a dominio per il nucleo dell'applicazione. È stato inoltre scelto lo stack tecnologico per realizzare l'applicazione.
- **Iterazione 1 (analisi, progettazione, sviluppo core):** durante questa fase ogni membro del gruppo ha portato avanti la realizzazione di un compito in particolare. Da un punto di vista dell'analisi sono stati formalizzati i comportamenti del sistema attraverso diagrammi di sequenza di sistema e contratti, nonché i diagrammi di stato e delle attività per definire rigorosamente le transizioni delle barriere e i flussi dell'utente. Passando allo sviluppo del nucleo, il lavoro è stato parallelizzato assegnando a ciascun componente un modulo indipendente: la creazione dello schema del database (Prisma) e l'interfacciamento con il backend (Supabase), l'implementazione dell'autenticazione utente con la struttura di routing (React Router v7), e la prima integrazione della mappa interattiva (Mapbox) dotata di geolocalizzazione. Sono stati realizzati in questa fase il setup della repository con l'installazione delle librerie necessarie, il setup dei servizi utilizzati e il recupero delle API key necessarie per il progetto.
- **Iterazione 2 (sviluppo funzionalità avanzate, testing e rilascio):** in questa seconda e ultima fase, il team si è concentrato sull'implementazione delle logiche di business più complesse e sul consolidamento della qualità del software. Dal punto di vista della progettazione dettagliata, sono stati redatti i diagrammi di sequenza di progettazione e il diagramma delle classi definitivo, parallelamente a un'analisi strutturale tramite il tool Understand che ha guidato un'analisi architetturale per la risoluzione degli antipattern. Lo sviluppo è proseguito in parallelo: sono state implementate le operazioni CRUD per la gestione delle segnalazioni, l'algoritmo di calcolo dei percorsi accessibili tramite OpenRouteService, con esclusione dinamica dei poligoni di ostacolo, il pannello di moderazione per l'Admin e il sistema di Notifiche Push integrato con Firebase. La logica di calcolo del Reputation Score e le relative transizioni di stato sono state automatizzate e messe in sicurezza tramite l'uso di Trigger SQL sul database. L'iterazione si è infine conclusa con la validazione dell'applicativo attraverso la stesura degli Unit Test (Vitest), la configurazione delle pipeline CI/CD (GitHub Actions) per il deploy automatico su Vercel e l'analisi statica continua del codice sorgente tramite SonarCloud.

Diagrammi di Gantt

I seguenti diagrammi di Gantt sono una pianificazione del lavoro da svolgere entro la settimana. Tutti i diagrammi di Gantt sono stati realizzati con Mermaid live editor.



Analisi dei Requisiti

Elenco attori e casi d'uso

Ho sostanzialmente tre attori:

- Utente non autenticato: è un utente non autenticato nel sistema
- Utente autenticato: è un utente autenticato nel sistema
- Admin: è un utente con ruolo di amministratore

L'applicazione distingue quindi due tipi di utenti autenticati: utenti normali e admin. L'utente admin svolge anche una funzione da moderatore. Il ruolo di admin deve essere assegnato esclusivamente dallo sviluppatore dell'app, in quanto un admin avrà il potere di modificare qualsiasi informazione inserita dagli utenti.

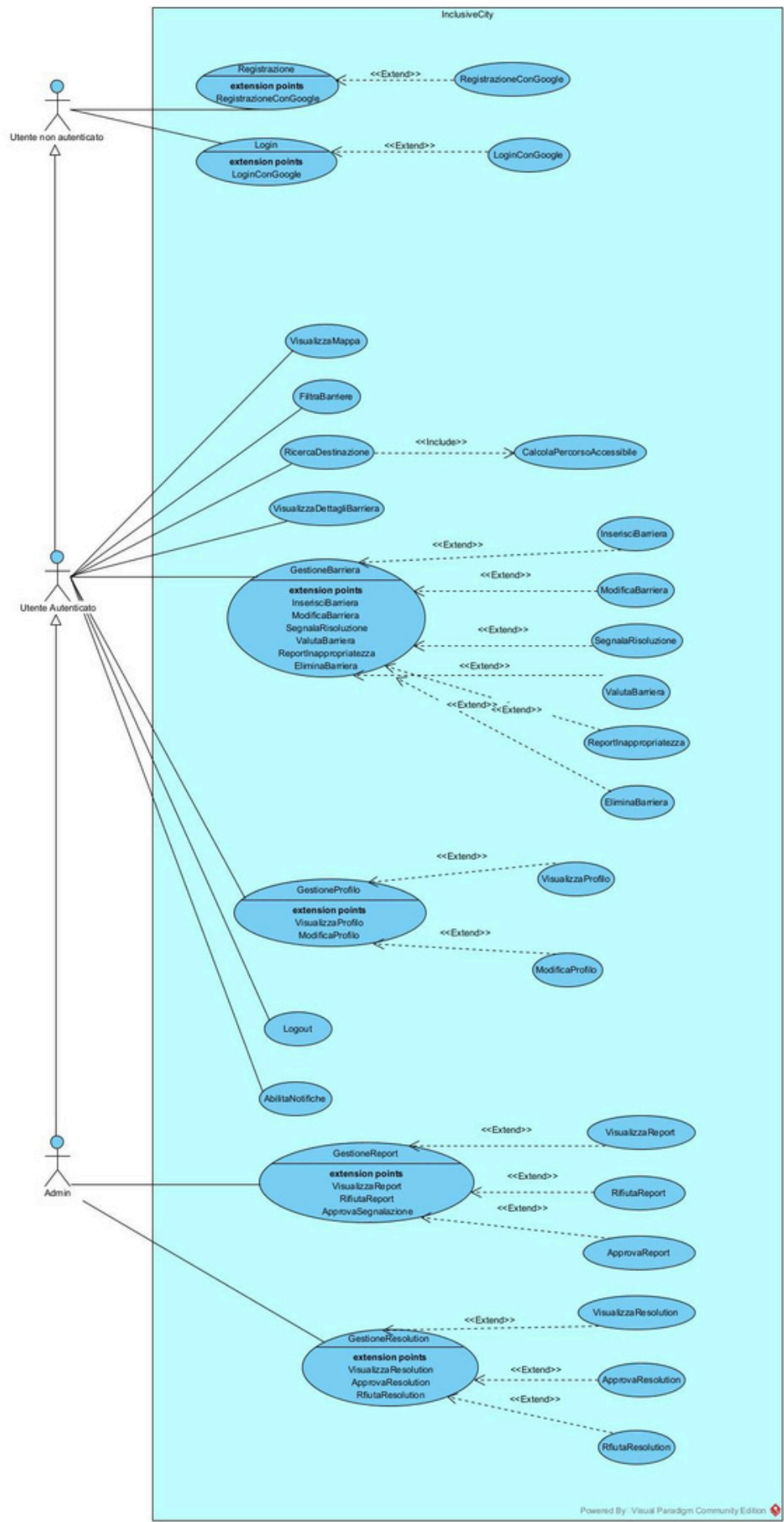
Un utente registrato può:

- effettuare login/logout e modificare i dati del profilo
- visualizzare la mappa con tutte le barriere, anche filtrandole per difficoltà
- ricercare un percorso adatto verso una meta
- visualizzare la lista intera di barriere, con la possibilità di filtrare per nome o stato
- inserire una nuova barriera
- modificare i dati o eliminare una barriera da lui creata
- segnalare la risoluzione di una barriera
- reportare una barriera per inappropriatezza
- dare un voto da 1 a 5 stelle con un commento opzionale a una barriera non creata da lui
- ricevere notifiche per segnalazioni a barriere create da lui

Un utente amministratore può (oltre alle azioni dell'utente):

- visualizzare la lista delle barriere con degli avvisi di risoluzione
- approvare manualmente la risoluzione di una barriera
- visualizzare la lista delle barriere con dei report
- nascondere manualmente una barriera

Diagramma dei Casi d'Uso



Descrizione dettagliata dei casi d'uso principali

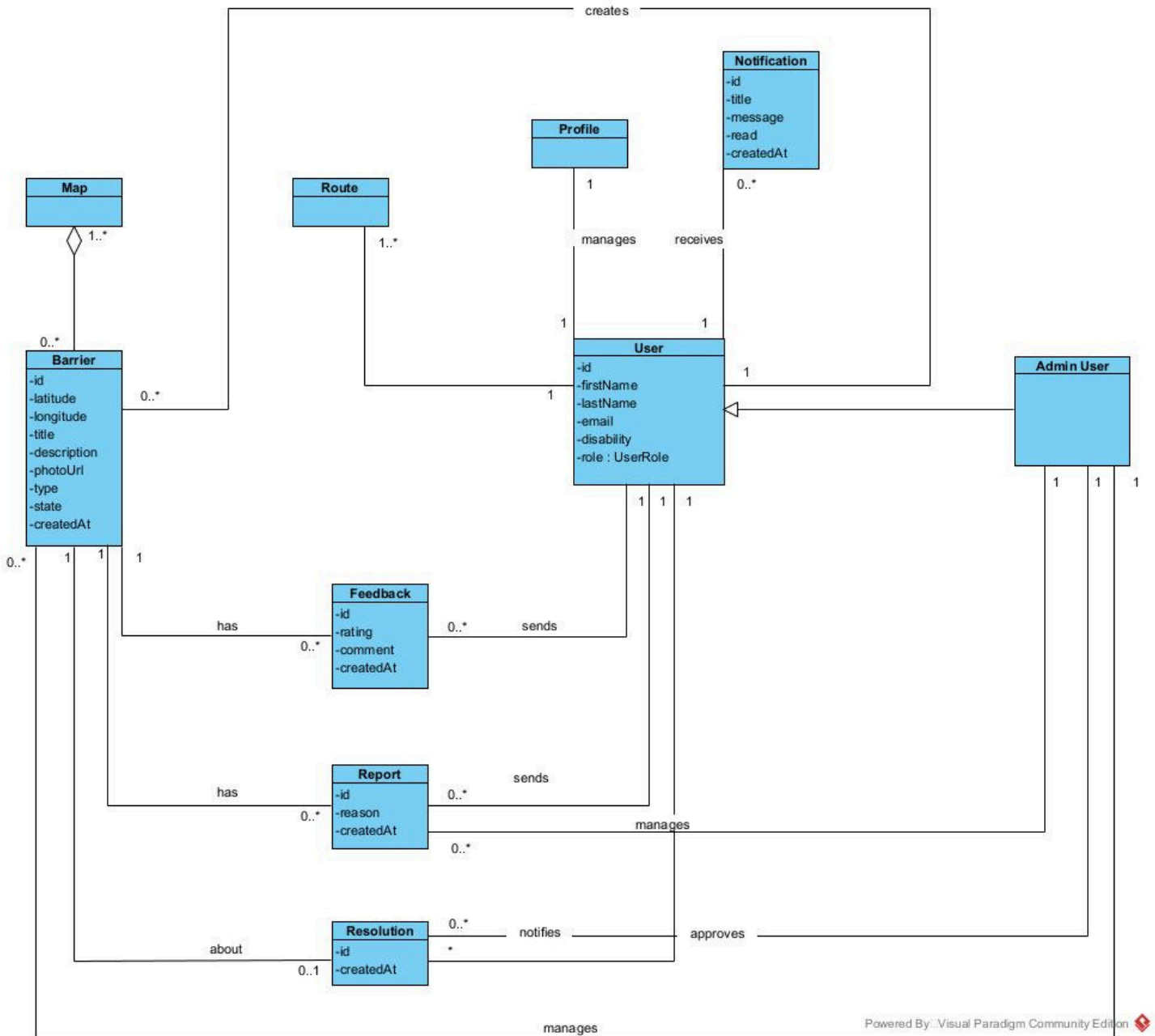
VisualizzaMappa	
Nome	VisualizzaMappa
Portata	Sistema InclusiveCity
Livello	Obbiettivo utente
Attore primario	Utente autenticato
Parti interessate e interessi	● Utente: vuole visualizzare le barriere architettoniche nella propria area per pianificare i propri spostamenti.
Pre-condizioni	● L'applicazione è avviata. ● La connessione di rete è disponibile.
Post-condizioni	● Viene visualizza la mappa centrata sulla posizione corrente o di default. ● Sono stati inseriti i markers delle barriere visualizzabili nell'area mostrata nella mappa.
Scenario principale di successo	1. L'utente accede alla schermata principale dell'applicazione. 2. Il sistema carica la mappa centrata sulla posizione corrente dell'utente. 3. Il sistema recupera le barriere attive presenti nell'area visibile. 4. Il sistema mostra i segnaposto delle barriere sulla mappa. 5. L'utente sposta la mappa o modifica lo zoom. 6. Il sistema aggiorna dinamicamente le barriere visibili in base alla nuova area. <i>Ripetere passi 5 e 6 tante volte quante l'utente richiede</i>
Estensioni	● 2a - Posizione corrente non disponibile: il sistema centra la mappa su una posizione di default. ● 3a - Nessuna barriera presente nell'area: il sistema mostra la mappa senza segnaposto.
Requisiti speciali	La mappa deve aggiornarsi in tempo reale al variare dell'area visibile.
Frequenza di utilizzo	Molto alta, è la schermata principale dell'applicazione.

RicercaDestinazione	
Nome	RicercaDestinazione
Portata	Sistema InclusiveCity
Livello	Obbiettivo utente
Attore primario	Utente autenticato
Parti interessate e interessi	• Utente: vuole raggiungere una destinazione seguendo un percorso accessibile in base alla propria disabilità.
Pre-condizioni	• La mappa è visualizzata. • L'utente ha accesso alla barra di ricerca o può selezionare un punto sulla mappa.
Post-condizioni	• È stata creata un'istanza di Destinazione a partire dall'input fornito. • È stata creata un'istanza di Percorso calcolata tenendo conto delle barriere e della disabilità dell'utente.
Scenario principale di successo	1. L'utente inserisce una destinazione nella barra di ricerca oppure seleziona un punto sulla mappa. 2. Il sistema interpreta l'input e crea un'istanza di Destinazione. 3. Il sistema include CalcolaPercorsoAccessibile per calcolare il percorso ottimale. 4. Il sistema mostra il percorso accessibile sulla mappa.
Estensioni	• 2a - Destinazione non valida o non riconosciuta: il sistema mostra un messaggio di errore e richiede un nuovo input. • 3a - Nessun percorso completamente accessibile disponibile: il sistema propone il percorso migliore possibile e segnala le barriere critiche presenti.
Requisiti speciali	Il calcolo del percorso deve tenere conto del tipo di disabilità dell'utente autenticato, se presente.
Frequenza di utilizzo	Alta, è lo scopo principale dell'applicazione.

GestioneBarriera	
Nome	GestioneBarriera
Portata	Sistema InclusiveCity
Livello	Obbiettivo utente
Attore primario	Utente autenticato
Parti interessate e interessi	● Utente autenticato: vuole inserire, modificare, eliminare, valutare o segnalare una barriera architettonica. ● Altri utenti: hanno interesse che le barriere siano accurate e aggiornate.
Pre-condizioni	● L'utente è autenticato. ● L'utente ha accesso alla mappa o alla lista delle barriere.
Post-condizioni	● L'attributo o lo stato della Barriera coinvolta è stato aggiornato in base all'operazione eseguita (gestito nel caso d'uso specifico esteso).
Scenario principale di successo	1. L'utente accede alla sezione barriere o seleziona una barriera dalla mappa. 2. Il sistema mostra il dettaglio della barriera con le operazioni disponibili. 3. L'utente seleziona una delle operazioni: InserisciBarriera, ModificaBarriera, EliminaBarriera, SegnalaRisoluzione, ValutaBarriera, ReportInappropriatezza. 4. Il sistema estende il caso d'uso specifico selezionato. 5. Il sistema aggiorna i dati della barriera in base all'operazione eseguita.
Estensioni	● 3a - L'utente non è il proprietario della barriera: il sistema non mostra le opzioni ModificaBarriera, EliminaBarriera, SegnalaRisoluzione. ● 3b - La barriera è già in stato RESOLVED o HIDDEN: il sistema disabilita le operazioni non applicabili allo stato corrente.
Requisiti speciali	Le operazioni disponibili devono variare dinamicamente in base al ruolo dell'utente e allo stato della barriera.
Frequenza di utilizzo	Alta.

GestioneSegnalazioni	
Nome	GestioneSegnalazioni
Portata	Sistema InclusiveCity
Livello	Obbiettivo utente
Attore primario	Admin
Parti interessate e interessi	• Admin: vuole revisionare le segnalazioni ricevute e mantenere la qualità dei dati presenti sulla mappa. • Utenti segnalatori: vogliono che le proprie segnalazioni vengano prese in carico. • Utenti autori delle barriere: vogliono essere informati dell'esito delle segnalazioni che li riguardano.
Pre-condizioni	• L'admin è autenticato. • Esistono istanze di Report o Resolution in stato PENDING.
Post-condizioni	• L'attributo status della Segnalazione (Report o Resolution) è stato aggiornato a REVIEWED, APPROVED o DISMISSED. • L'attributo state della Barriera associata è stato aggiornato di conseguenza, se applicabile. • L'attributo reputationScore degli utenti coinvolti è stato aggiornato secondo le regole di moderazione.
Scenario principale di successo	1. L'admin accede alla sezione "Segnalazioni". 2. Il sistema mostra l'elenco delle segnalazioni in stato PENDING. 3. L'admin seleziona una segnalazione. 4. Il sistema mostra i dettagli della segnalazione e della barriera associata. 5. L'admin sceglie di approvare o rifiutare la segnalazione. 6. Il sistema aggiorna lo stato della segnalazione. 7. Il sistema aggiorna lo stato della barriera associata, se necessario. 8. Il sistema aggiorna il reputationScore degli utenti coinvolti secondo le regole di moderazione.
Estensioni	• 5a - L'admin rifiuta la segnalazione: il sistema imposta lo stato della segnalazione a DISMISSED e non modifica la barriera. Il reputationScore degli autori viene aggiornato di conseguenza. • 8a - La segnalazione riguarda inapproprietezza approvata: il sistema estende EliminaQualsiasiBarriera per nascondere la barriera dalla mappa.
Requisiti speciali	Il sistema deve mostrare contestualmente i dettagli della barriera associata per agevolare la decisione dell'admin.
Frequenza di utilizzo	Media, dipende dal volume di segnalazioni ricevute.

Modello di Dominio



Le entità fondamentali del dominio rilevate sono le seguenti:

- **Disability:** tipo di disabilità.
- **User:** utente registrato nel sistema.
- **Barrier:** segnalazione di una barriera architettonica da parte di un utente.
- **Feedback:** voto (da 1 a 5 stelle) con commento opzionale dato da un utente ad una barriera.
- **Report:** report effettuato da un utente ad una barriera in quanto errata o inappropriata.
- **Resolution:** notifica (con prova fotografica) di risoluzione di una barriera.
- **Notification:** messaggio per l'utente.

Comportamento del Sistema

Diagramma di Stato Barriera, reputation score, moderazione automatica e manuale

Una barriera può trovarsi nei seguenti stati logici:

- **ACTIVE:** è attiva, viene mostrata sulla mappa
- **RESOLVED:** è stata correttamente risolta, non rappresenta più un ostacolo e quindi non viene mostrata sulla mappa
- **IN_REVIEW:** la barriera è segnalata come in revisione, viene mostrata sulla mappa ma con un indicatore di incertezza
- **HIDDEN:** la barriera è nascosta dalla mappa a causa di inappropriatezza o falsa segnalazione

I passaggi di stato di una barriera possono avvenire per due motivi: vengono fatti da un admin oppure dalla moderazione automatica.

Il reputation score ha un ruolo fondamentale per controllare il peso di un utente nelle segnalazioni. Un utente può guadagnare reputation score principalmente:

- inserendo una nuova barriera: +2
- inserendo un Feedback: +1
- con Resolution e Reports (vedi tabelle sotto)

È inoltre necessario specificare un dettaglio importante: in base al reputationScore di un utente, una Resolution o un Report può valere:

- $rs < 0$: 0 (utente non affidabile, non ha senso fidarsi delle sue segnalazioni)
- $0 \leq rs < 20$: 1 (nuovo utente)
- $20 \leq rs < 50$: 2 (utente abituale)
- $rs \geq 50$: 3 (utente veterano)

Definiamo il valore soglia del passaggio di stato come la somma del peso degli utenti che hanno proposto una Resolution o un Report.

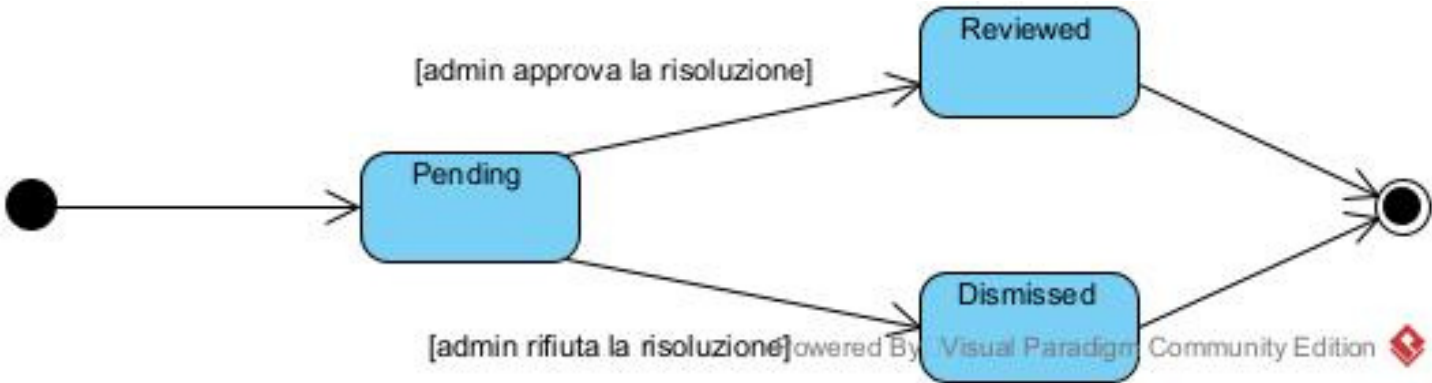
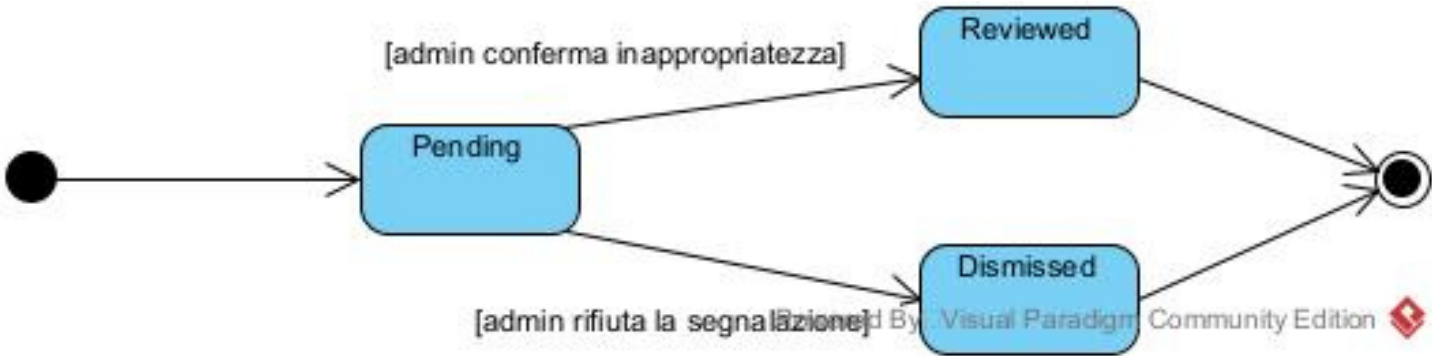
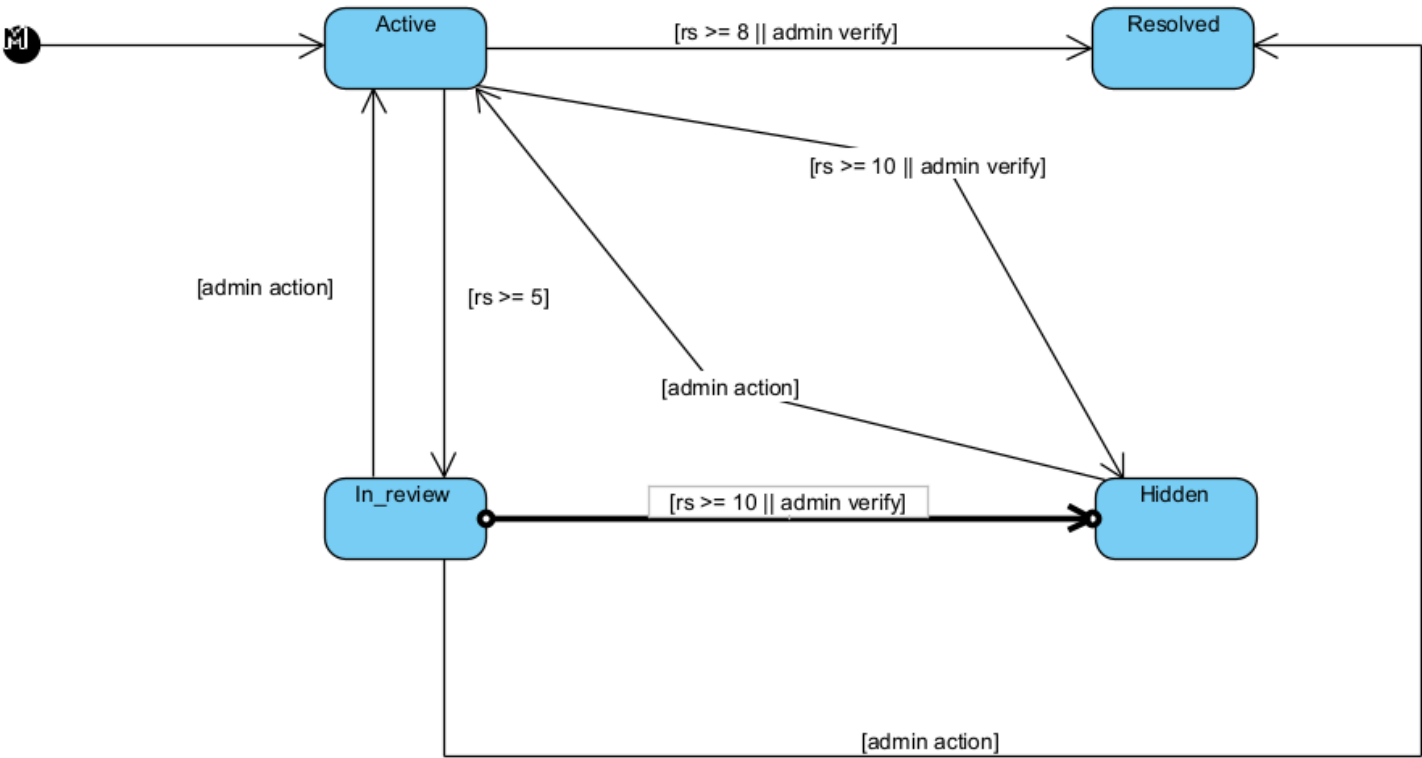
I passaggi dovuti alla moderazione automatica sono i seguenti. Teniamo conto nella moderazione automatica anche del reputation score di ogni utente.

PASSAGGIO	VALORE SOGLIA	EFFETTI RECORD	REPUTATION SCORE
ACTIVE → RESOLVED	8	tutte le Resolution passano da PENDING a APPROVED	autori Resolution +5
ACTIVE → IN_REVIEW	5	i Report restano in stato PENDING	no per evitare spam
ACTIVE/IN_REVIEW → HIDDEN	10	tutti i Report passano da PENDING a REVIEWED	autori Report +3 autore Barrier -5

L’admin può decidere in qualsiasi momento di effettuare delle modifiche allo stato di una barriera. Tutte e sole le transizioni consentite all’admin sono le seguenti:

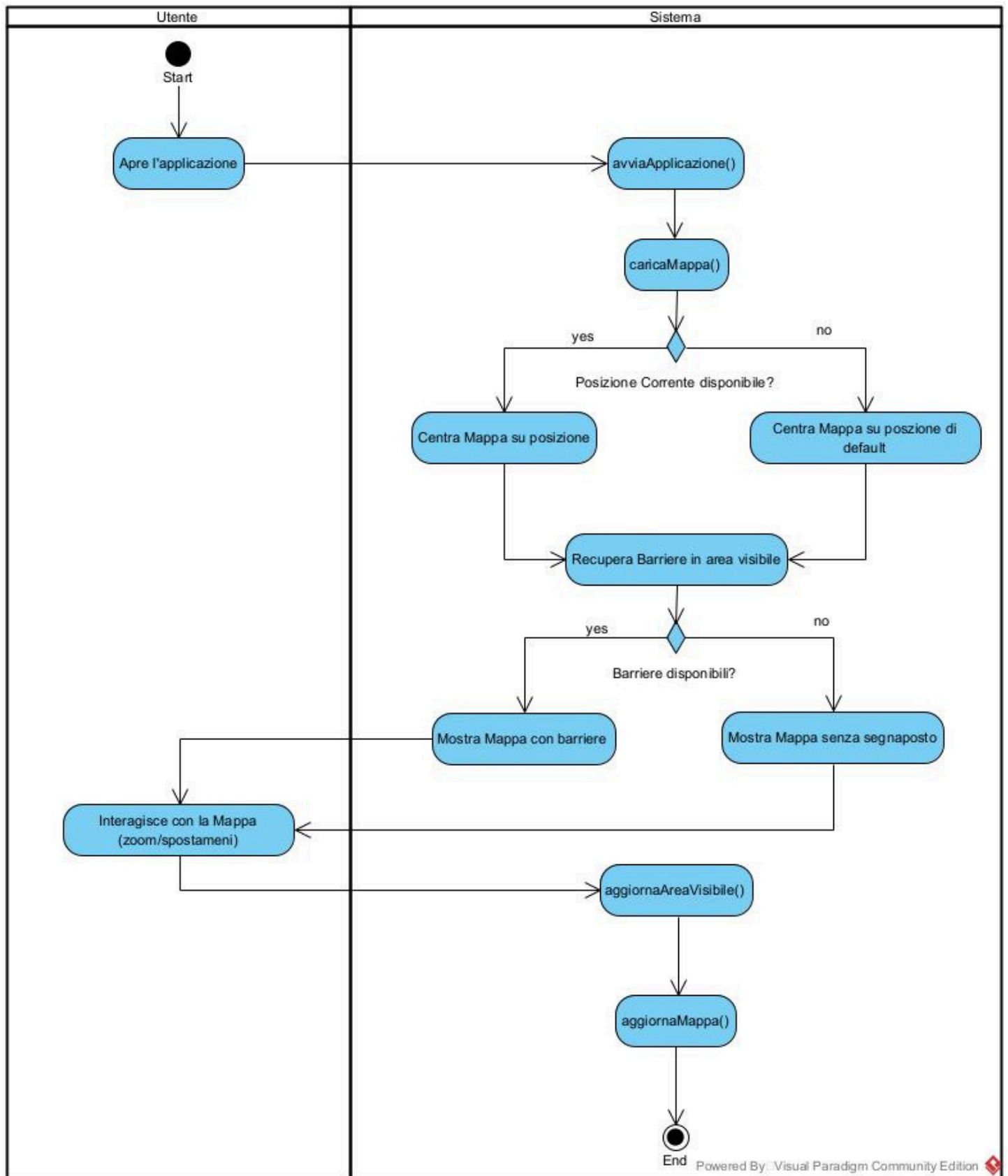
PASSAGGIO	CONDIZIONE	EFFETTI RECORD	REPUTATION SCORE
IN_REVIEW → HIDDEN	L’admin conferma che la barriera è inappropriata	i Report passano da PENDING a REVIEWED	autori Report +3 autore Barrier -5
IN_REVIEW → ACTIVE	L’admin respinge le segnalazioni	i Report passano da PENDING a DISMISSED	autori Report -3
ACTIVE/IN_REVIEW → RESOLVED	L’admin verifica la risoluzione di una barriera	la Resolution passa da PENDING a APPROVED con approverId	autore Resolution +5
x	L’admin rifiuta la risoluzione di una barriera	la Resolution passa da PENDING a REJECTED	autore Resolution -3
ACTIVE → HIDDEN	L’admin nota una barriera inappropriata e la nasconde	i Report passano da PENDING a REVIEWED	autore Barrier -5

Lo schema dei passaggi di stato delle barriere è il seguente:

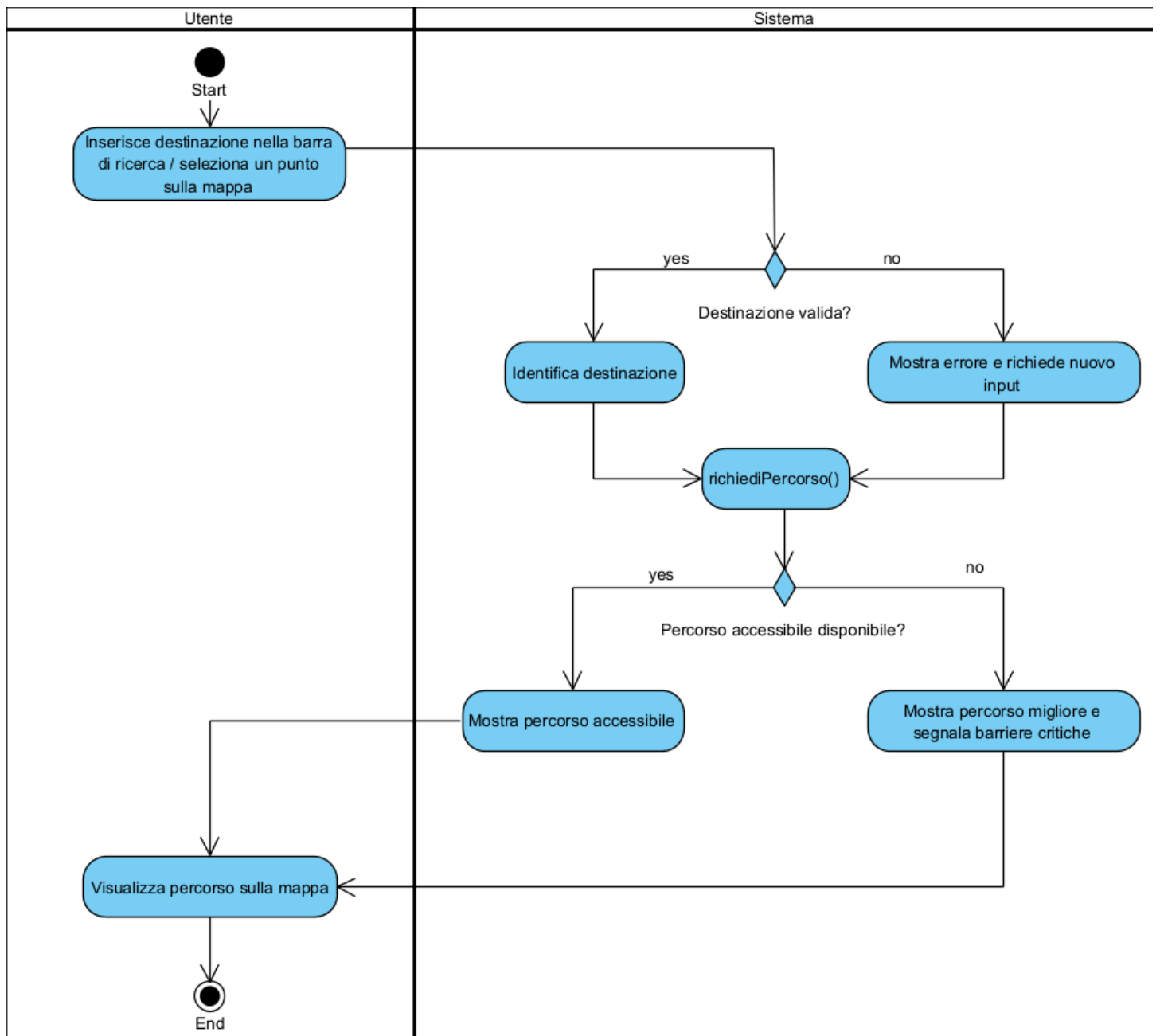


Diagrammi di Attività

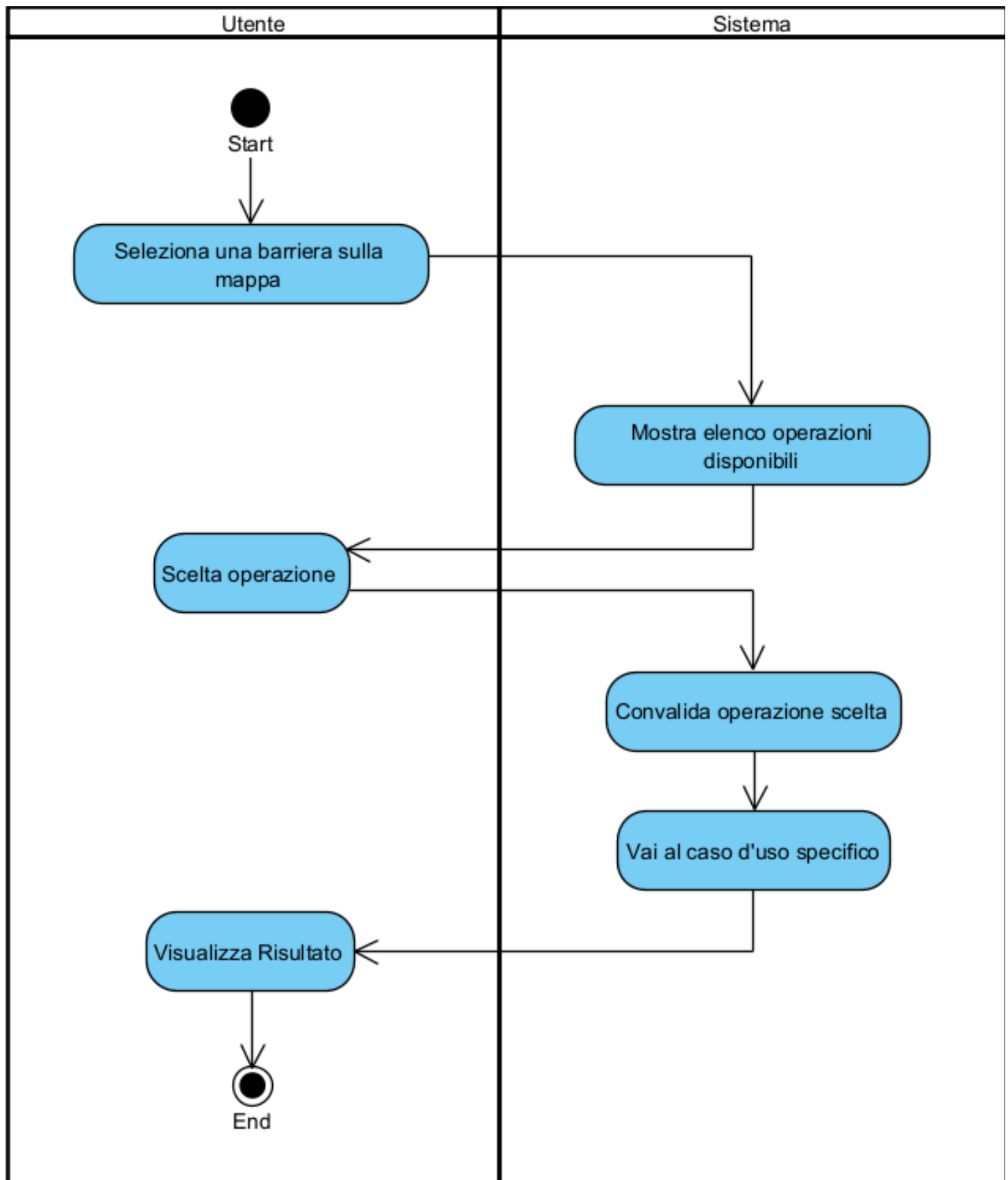
1. VisualizzaMappa



2. RicercaDestinazione



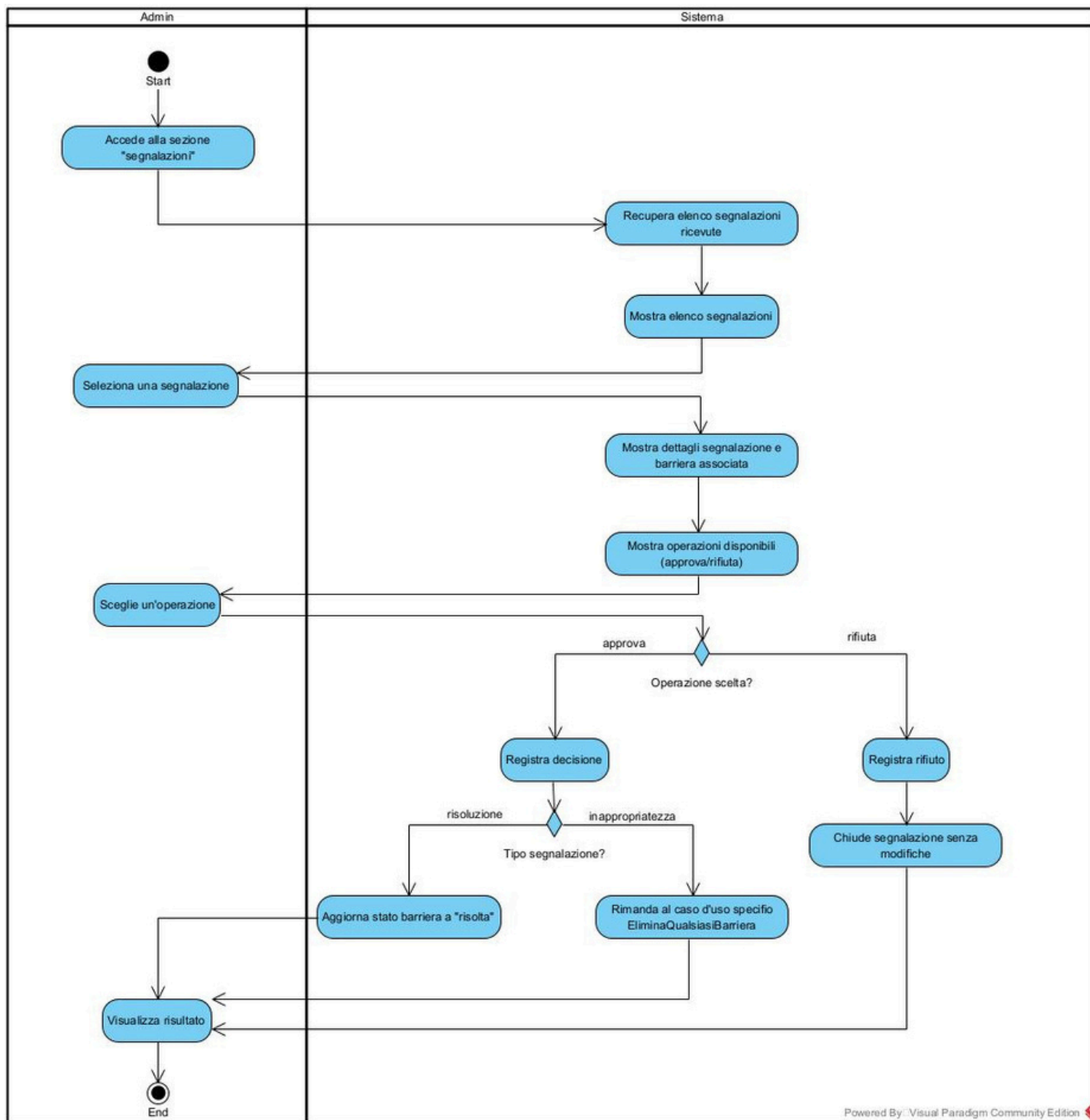
3. GestioneBarriera



Nota

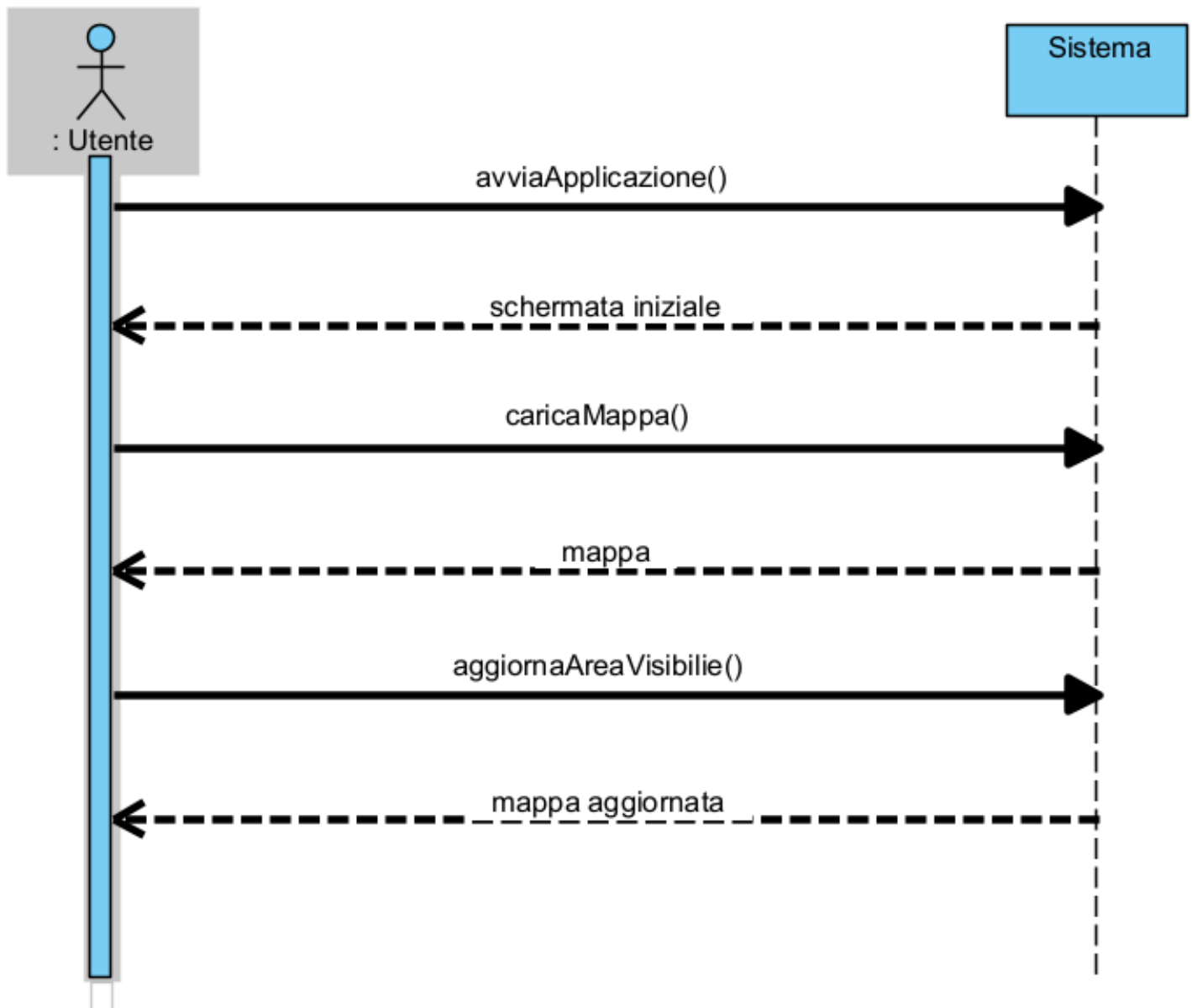
Questo è solo il caso d'uso generale, le operazioni specifiche vengono rimandate al caso d'uso specifico per la singola operazione che si sceglie di fare.

4. GestioneSegnalazioni



Diagrammi di Sequenza di Sistema (SSD) e Contratti

1. VisualizzaMappa



Contratto operazione avviaApplicazione()

Responsabilità	Inizializzare il sistema e prepararlo alla visualizzazione della schermata iniziale.
Pre-condizioni	• L'utente è autenticato correttamente nel sistema.
Post-condizioni	• Il sistema è in stato "applicazione avviata".

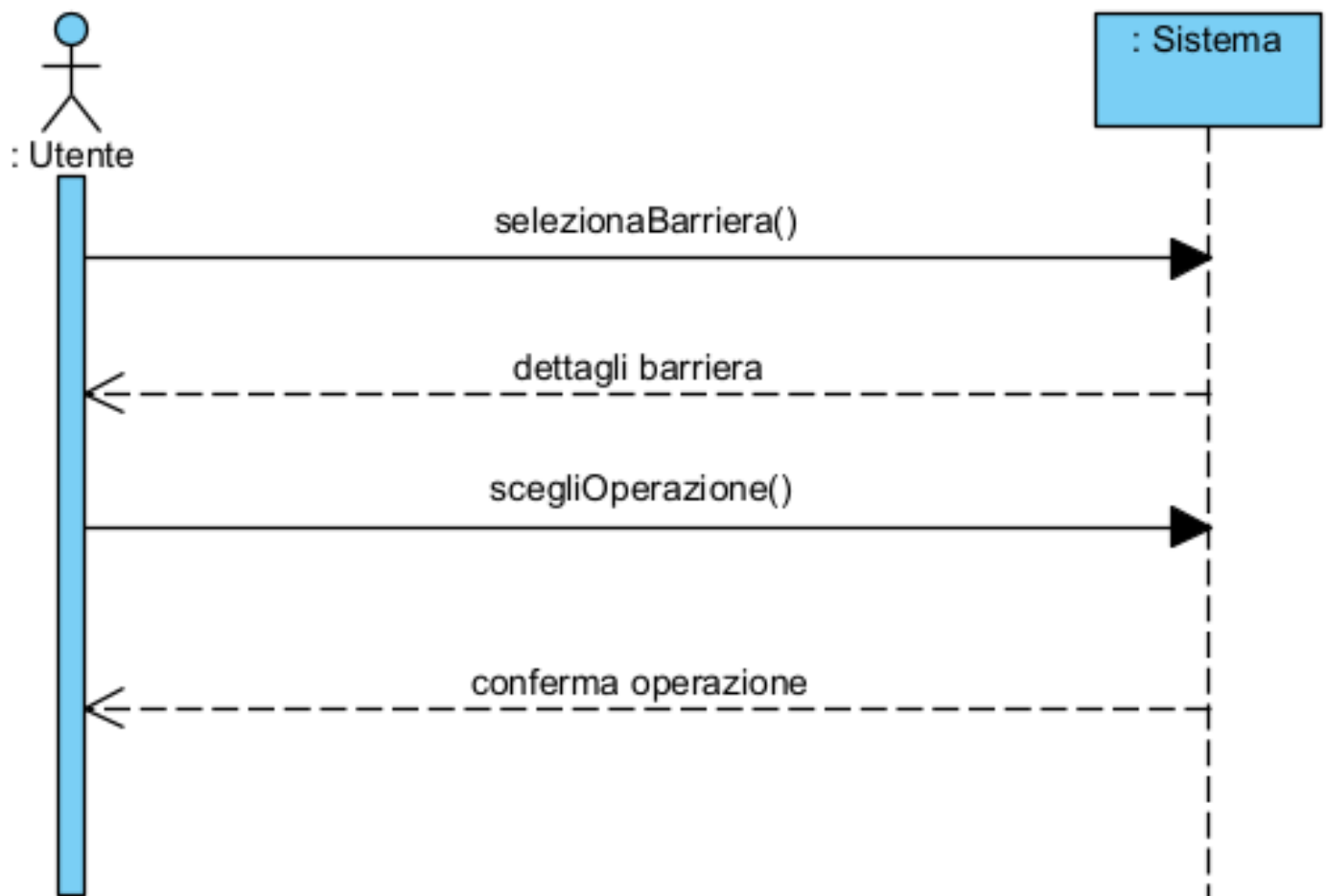
Contratto operazione caricaMappa()

Responsabilità	Caricare la mappa centrata sulla posizione corrente dell'utente.
Pre-condizioni	• L'applicazione è stata avviata. • La posizione corrente dell'utente è disponibile.
Post-condizioni	• È stata creata un'istanza di Mappa con posizione centrata sulla posizione corrente. • L'attributo areaVisibile di Mappa è stato inizializzato. • È stata formata un'associazione tra Mappa e le istanze di Barriera presenti in areaVisibile.

Contratto operazione aggiornaAreaVisibile()

Responsabilità	Aggiornare l'area visibile in base allo spostamento/zoom dell'utente.
Pre-condizioni	• Una istanza di mappa è già stata creata.
Post-condizioni	• L'attributo areaVisibile di Mappa è stato aggiornato. • Le associazioni tra Mappa e le istanze di Barriera sono state aggiornate in base alla nuova areaVisibile.

2. GestioneBarriera



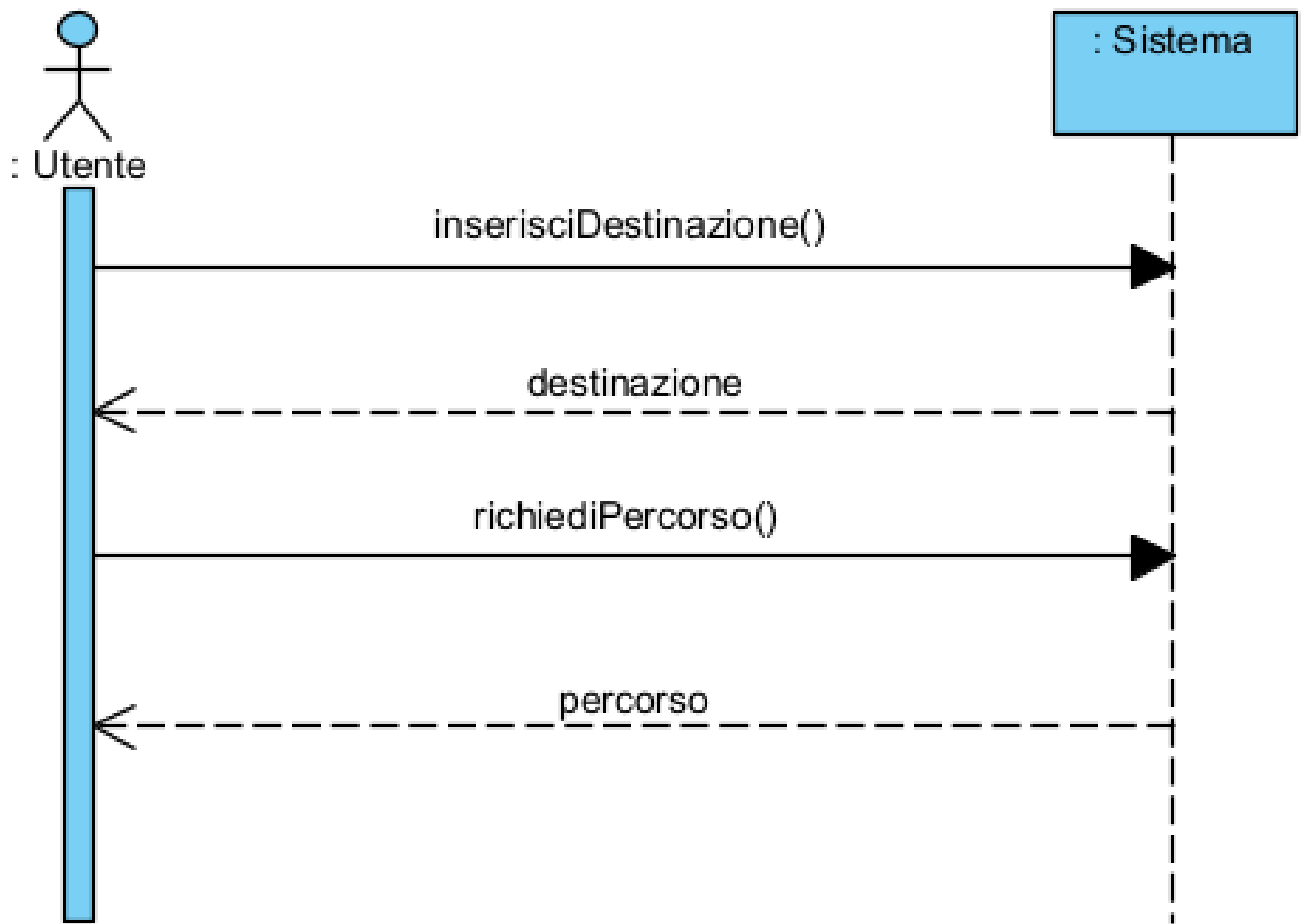
Contratto operazione selezionaBarriera()

Responsabilità	Recuperare le informazioni della barriera selezionata.
Pre-condizioni	• L'utente è autenticato correttamente nel sistema • Esiste un'istanza di Barriera con l'identificativo specificato.
Post-condizioni	• Creazione di un'istanza di associazione tra la barriera corrente e il sistema.

Contratto operazione scegliOperazione()

Responsabilità	Determinare l'azione richiesta dall'utente sulla barriera.
Pre-condizioni	• Una istanza di Barriera è associata come barrieraCorrente.
Post-condizioni	• L'attributo operazioneScelta del Sistema è stato impostato al valore di tipoOperazione. • E' stata creata un'istanza di associazione fra l'operazione scelta e la barrieraCorrente.

3. RicercaDestinazione



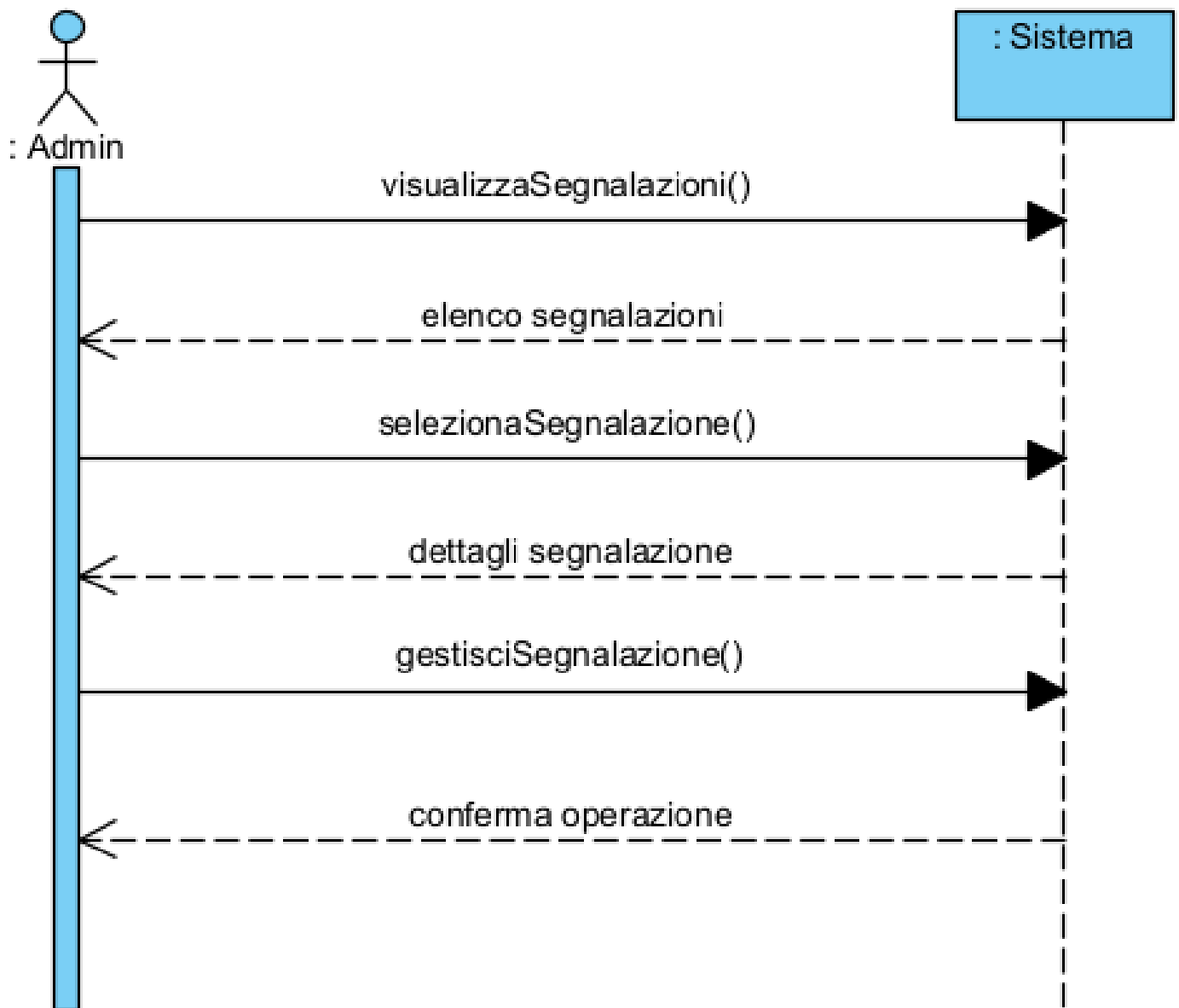
Contratto operazione inserisciDestinazione()

Responsabilità	Interpretare la destinazione inserita dall'utente e identificarla come punto valido.
Pre-condizioni	• Una istanza di mappa è visualizzata.
Post-condizioni	• È stata creata un'istanza di Destinazione a partire dall'input fornito. • È stata formata un'associazione tra il Sistema e l'istanza di Destinazione creata.

Contratto operazione richiediPercorso()

Responsabilità	Calcolare un percorso accessibile dalla posizione corrente alla destinazione.
Pre-condizioni	• È associata un'istanza di Destinazione al Sistema. • La posizione corrente dell'utente è nota.
Post-condizioni	• È stata creata un'istanza di Percorso calcolata a partire dalla posizione corrente e dalla Destinazione. • È stata formata un'associazione tra il Sistema e l'istanza di Percorso creata.

4. GestioneSegnalazioni



Contratto operazione `visualizzaSegnalazioni()`

Responsabilità	Recuperare tutte le segnalazioni presenti nel sistema.
Pre-condizioni	L'utente è autenticato e ha ruolo ADMIN.
Post-condizioni	Nessuna modifica al dominio.

Contratto operazione `selezionaSegnalazione()`

Responsabilità	Recuperare i dettagli della segnalazione scelta.
Pre-condizioni	Esiste un'istanza di Report o Resolution con l'identificativo specificato.
Post-condizioni	È stata formata un'associazione tra il Sistema e l'istanza di Segnalazione selezionata (<code>segnalazioneCorrente</code>).

Contratto operazione gestisciSegnalazione()

Responsabilità	Applicare l'azione scelta dall'admin (approvazione, rifiuto, chiusura).
Pre-condizioni	• Una istanza di Segnalazione è associata come segnalazioneCorrente. • L'admin ha scelto un'azione valida tra: APPROVA, RIFIUTA, NASCONDI.
Post-condizioni	• L'attributo status della Segnalazione è stato aggiornato in base all'azione scelta. • L'attributo state della Barriera associata è stato aggiornato di conseguenza. • L'attributo reputationScore degli utenti coinvolti è stato aggiornato secondo le regole di moderazione (autore Report, autore Resolution, autore Barriera). • Se l'azione è APPROVA su una Resolution: l'attributo approverId della Resolution è stato impostato all'id dell'admin.

Progettazione e Architettura

Ideazione schermate applicazione

Di seguito è presente una sezione dedicata esclusivamente alla progettazione delle schermate che l'applicazione avrà. È stato previsto anche un meccanismo per impedire l'accesso a sezioni specifiche: per esempio, non posso andare nel flusso di autenticazione se sono già autenticato.

Flusso/Schermata	Compito
Flusso di Autenticazione	● schermata login: ha i campi email/pwd, il tasto login con google ● schermata signin: ha i campi per la registrazione e il tasto login con google
Schermata di Onboarding	● fase tra signin e l'utilizzo effettivo dell'applicazione ● schermata di onboarding: fa scegliere la disabilità, viene mostrata solo tra registrazione e primo utilizzo dell'app. È un passo obbligatorio. La disabilità di un utente non potrà più essere cambiata
Schermata mappa	● mappa full screen con marker delle barriere ● barra di ricerca, tasto filtra barriere ● controlli mappa, tasto nuova barriera
Schermate barriere	● lista barriere: mostra tutte le barriere nella zona. cliccando su una barriera si apre la schermata di dettaglio (mostra tutte le informazioni). Posso filtrare con dei tag o con una barra di ricerca ● dettaglio barriera: mostra tutte le informazioni della barriera. Se sono l'autore mostra i tasti per modifica ed eliminazione. Se non sono l'autore mi permette di dare un feedback o di segnalare. Posso sempre proporre una prova di risoluzione ● nuova barriera: contiene il form per l'inserimento di una nuova barriera, compreso il caricamento di un'immagine e una mini mappa per precisare la posizione ● modifica barriera: contiene un form per la modifica barriera (come quello di inserimento)
Schermata del profilo	● dettagli profilo, modifica profilo ● pagina notifiche ● sezione le mie barriere/tutte le barriere ● se sono admin, un tasto che mi porta alla pagina admin ● tasto logout
Schermata notifiche	● lista delle notifiche ricevute dall'utente
Schermata admin	● lista delle risoluzioni, con possibilità di approvare o rifiutare ● lista delle segnalazioni, con possibilità di ignorare la segnalazione o nascondere la barriera

Stack Tecnologico

Frontend (Web, PWA):

- Framework: React Router v7
- Linguaggio: Typescript
- Build: Vite + vite-plugin-pwa
- UI: Tailwind CSS
- State management: React Context (per autenticazione), React Router Loader/Actions
- Form: React Hook Form, Zod (validazione)
- Hosting: Vercel

Mappe e geolocalizzazione

- UI: Mapbox GL JS
- Database GIS: PostGIS
- Routing: OpenRouteService API
- Geocoding: Mapbox Geocoding API
- Geolocalizzazione: Browser Geolocation API

Backend Supabase/Firebase:

- Database: PostgreSQL + GIS (supabase)
- ORM: Prisma v5 (supporto a GIS)
- Authentication: Supabase Auth
- Storage: Supabase Storage
- Notifications: Firebase Cloud Messaging
- Serverless Logic: Supabase Edge Functions (con Deno, per notifiche)

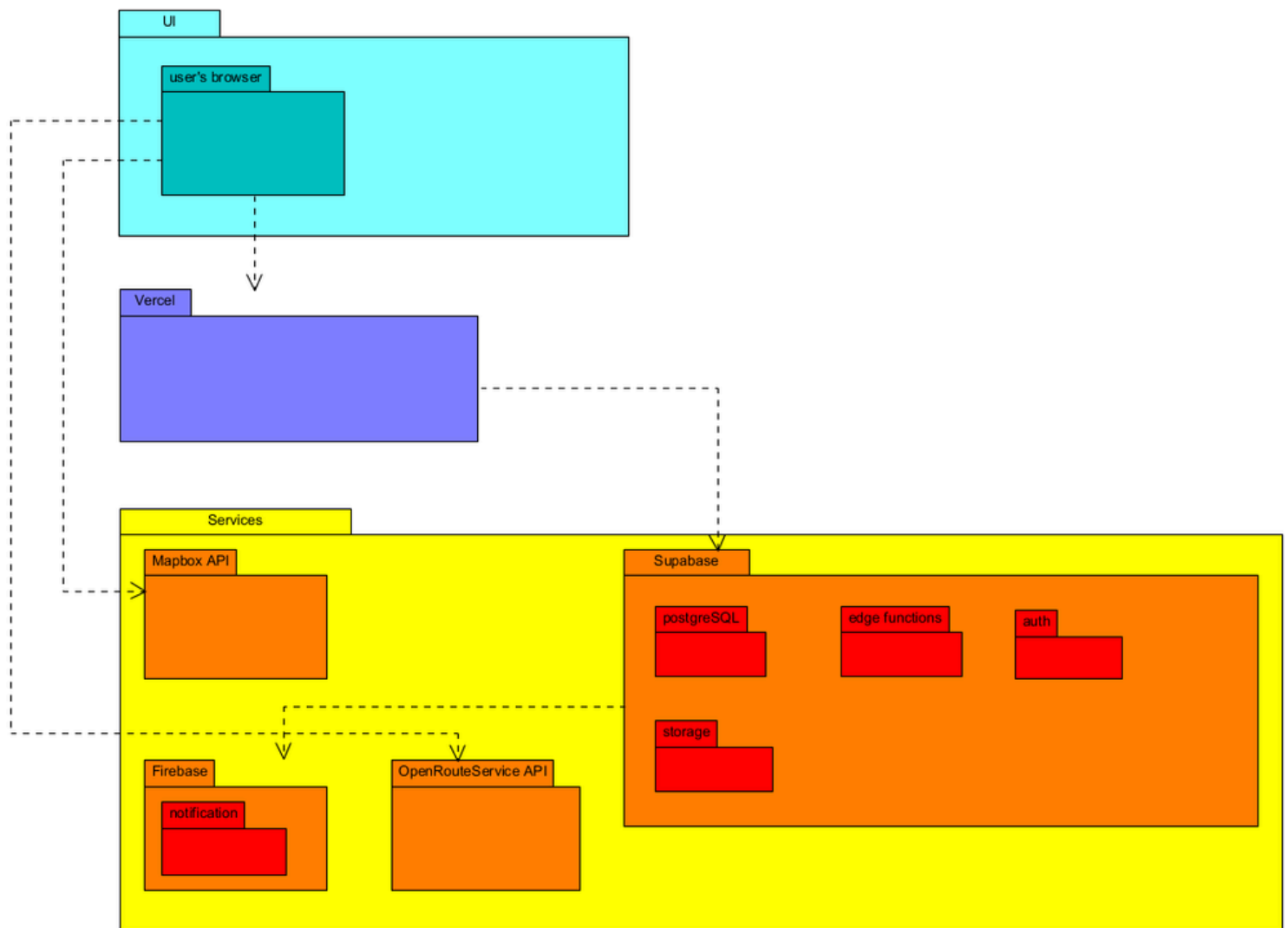
Test, DevOps

- Testing unitario: Vitest
- Code quality: SonarCloud
- CI: tsc, build test (Github Actions)
- CD: deploy automatico su Vercel (Github Actions)
- Versionamento: Git & Github

Documentazione:

- Diagrammi di analisi e progettazione: Visual Paradigm CE
- Diagrammi di Gantt: Mermaid live editor
- Documento di Analisi e progettazione: Canva

Diagramma dell'Architettura Software



Il diagramma mostra l'architettura dell'applicazione InclusiveCity.

Il pattern applicato è quello Client-Server, ma sono stati aggiunti elementi BaaS (Backend-as-a-Service) e API esterne.

Il client interagisce con Vercel, che esegue il server web, e direttamente con moduli dei servizi come auth e storage di supabase e notifications di firebase. Il client comunica inoltre direttamente con le api di Mapbox e OpenRouteService.

Vercel comunica con supabase per l'accesso ai dati mediante l'ORM Prisma.

Inoltre le edge functions di supabase comunicano con firebase per l'invio delle notifiche push.

È stato deciso di introdurre degli enum, in particolare:

- **UserRole:** indica il ruolo dell'utente
- **BarrierState:** indica lo stato della barriera
- **ReportReason:** indica il motivo del report
- **ReportStatus:** indica lo stato del report
- **ResolutionStatus:** indica lo stato della segnalazione di risoluzione
- **NotificationType:** indica il motivo della notifica

Il diagramma delle classi ottenuto è il seguente:

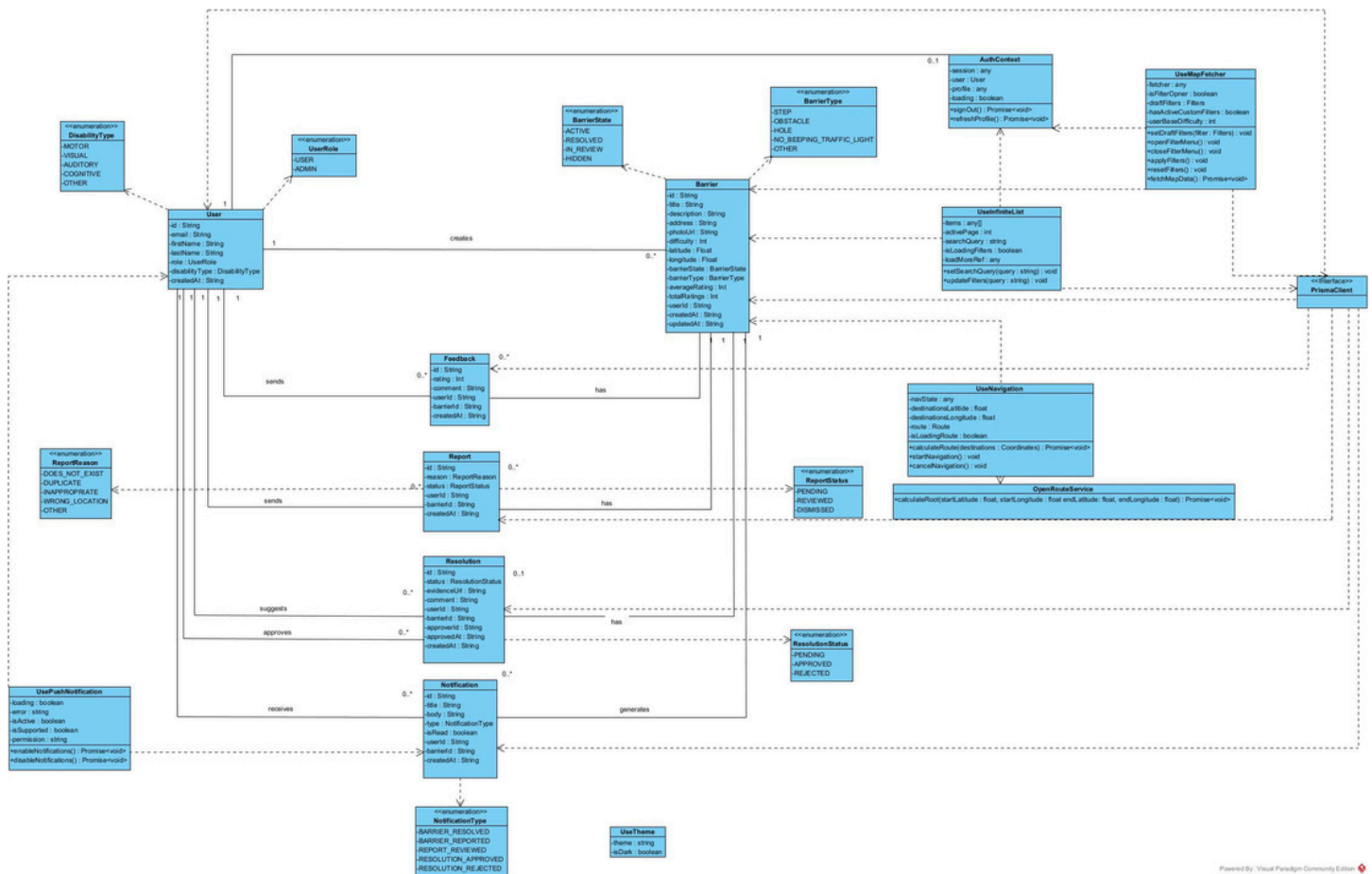
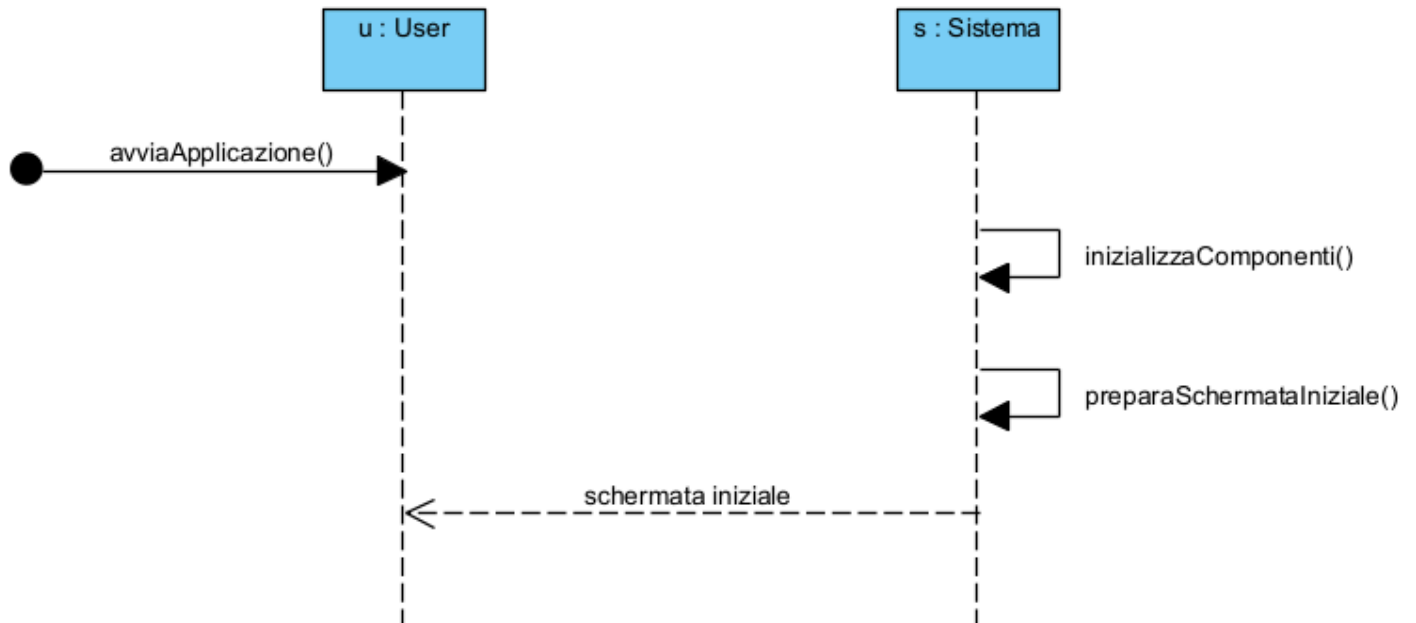


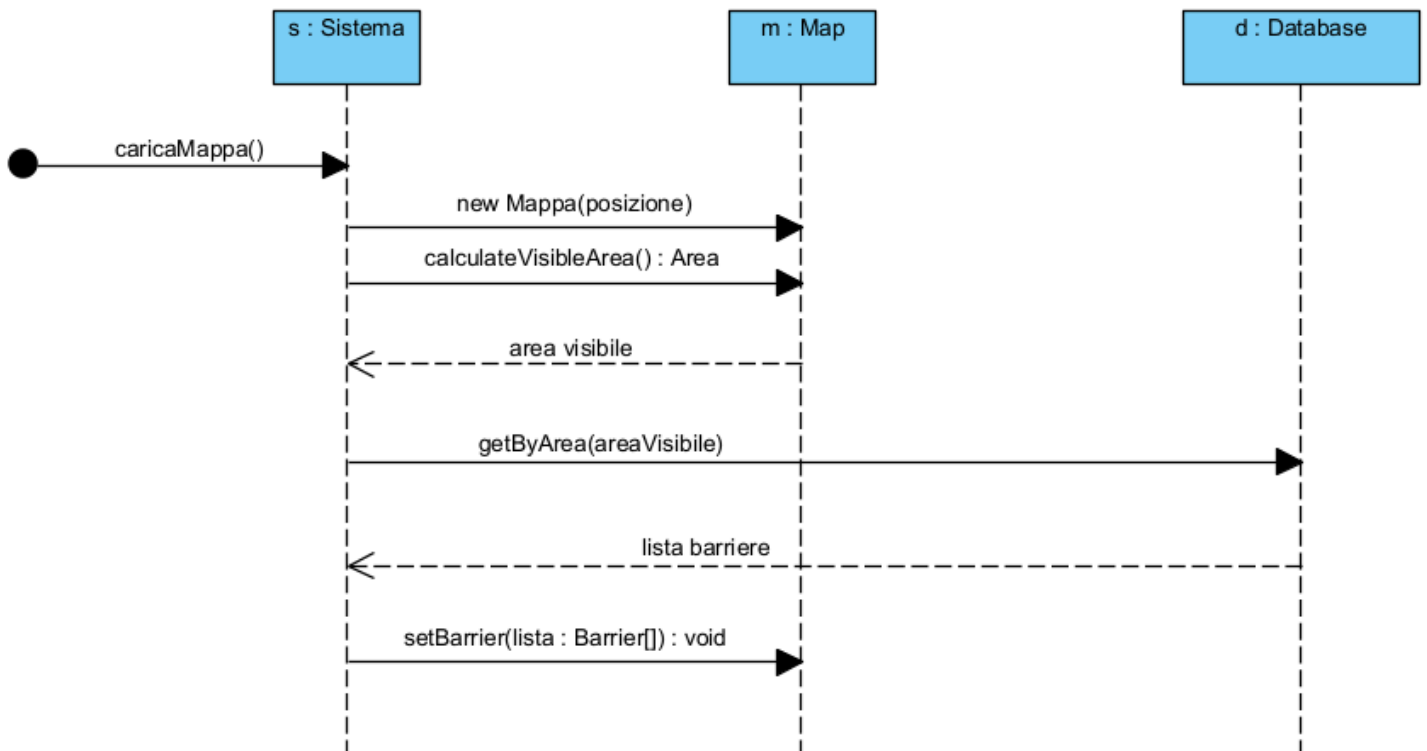
Diagramma di Sequenza

Sezione 1 - Visualizza Mappa

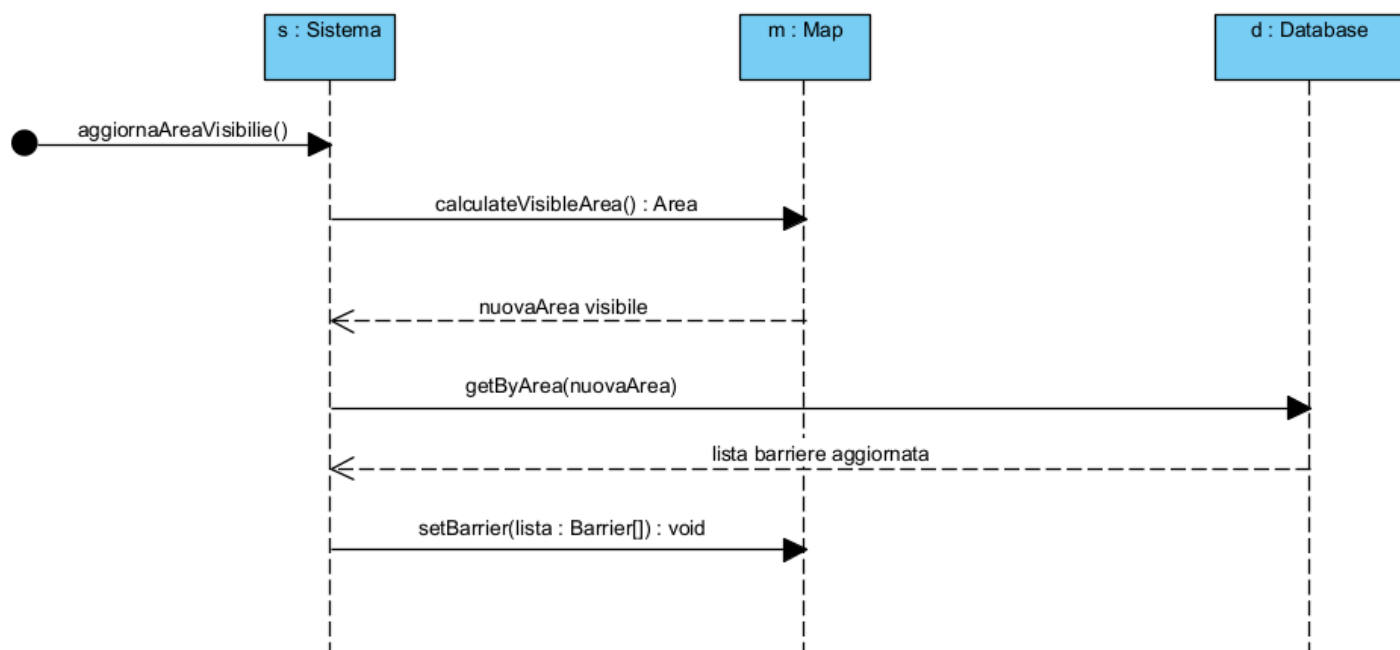
- avviaApplicazione()



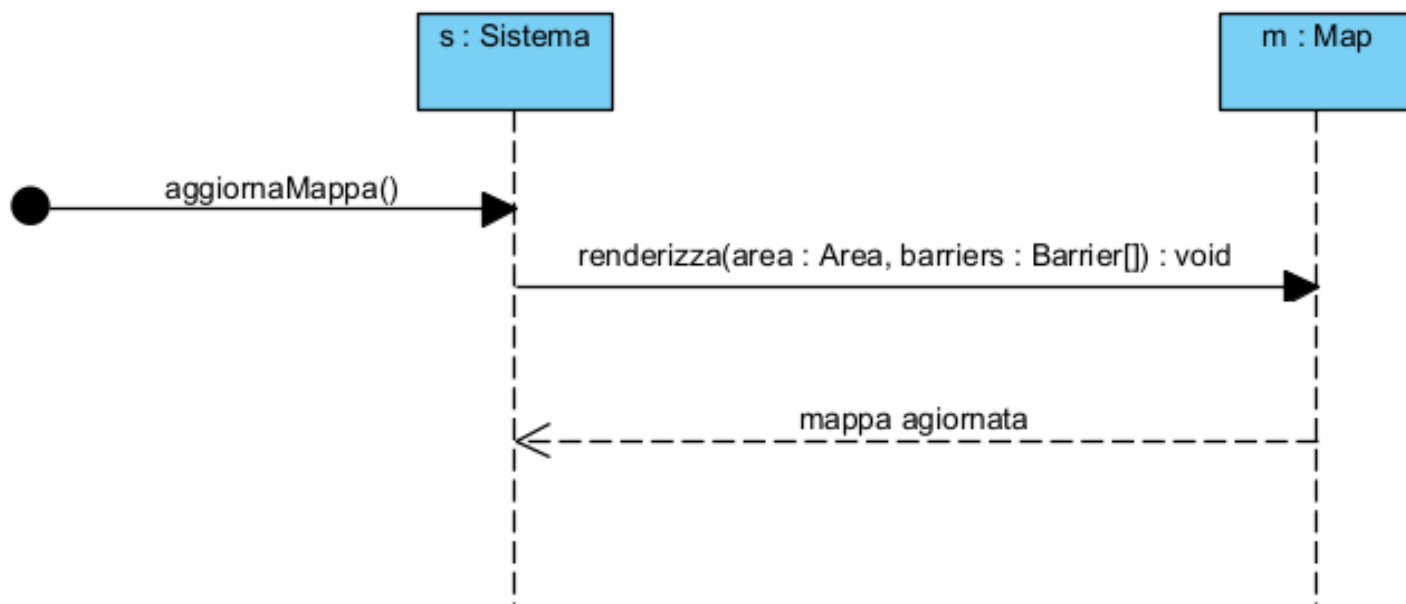
- caricaMappa()



- **aggiornaAreaVisibile()**

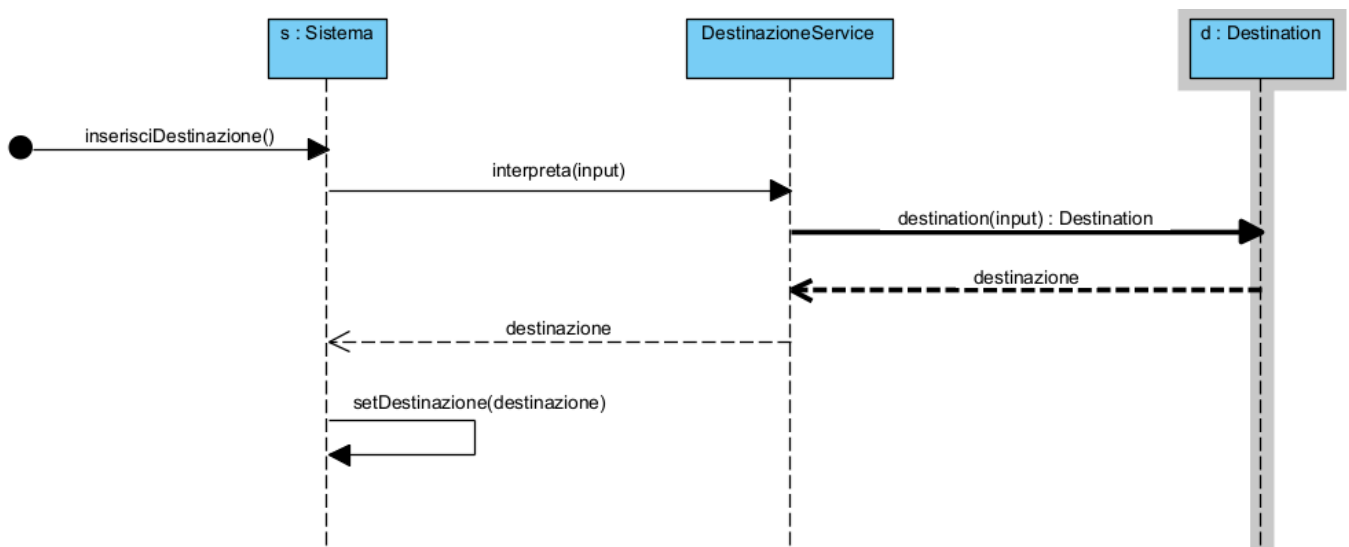


- **aggiornaMappa()**

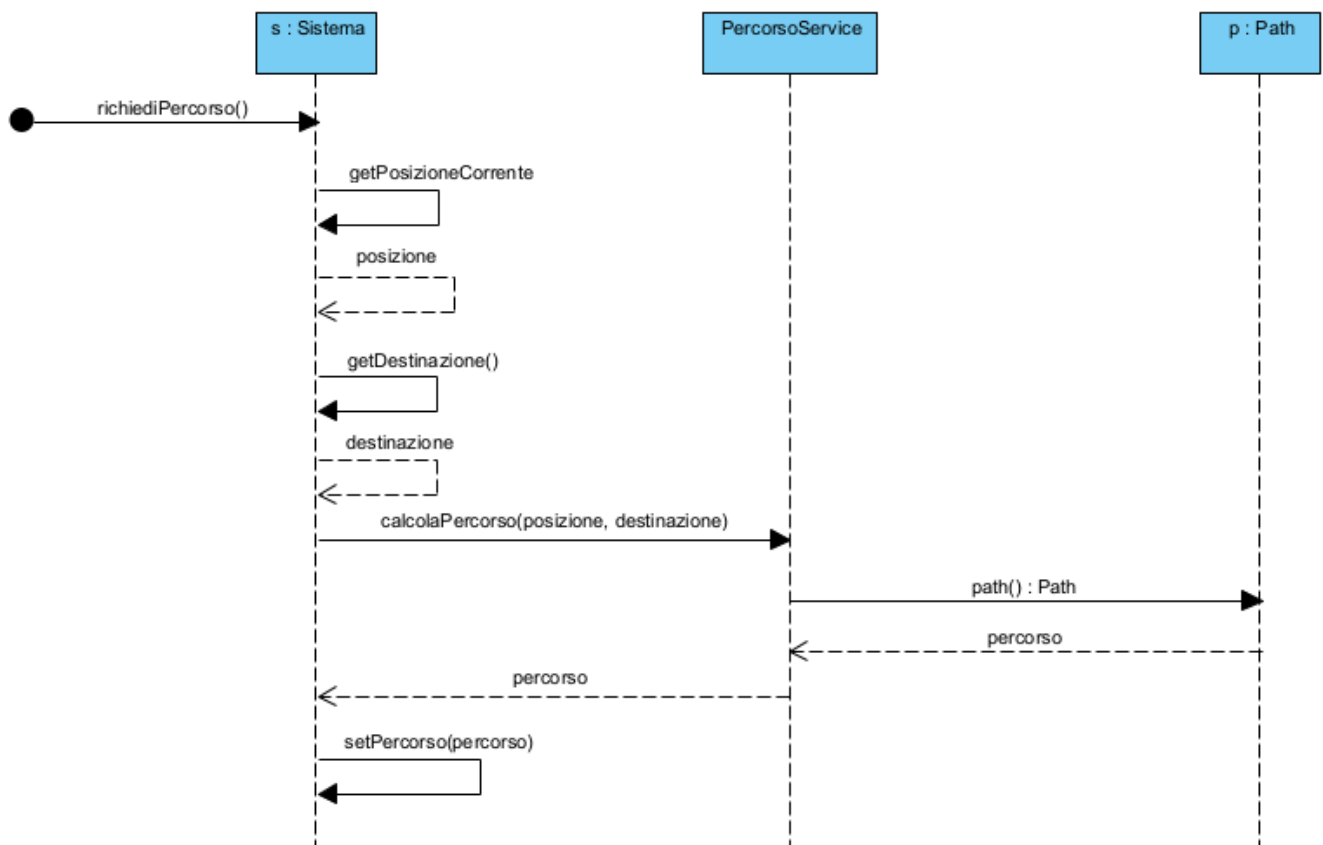


SEZIONE 2 - Ricerca Destinazione

- **inserisciDestinazione()**



- **richiediPercorso()**



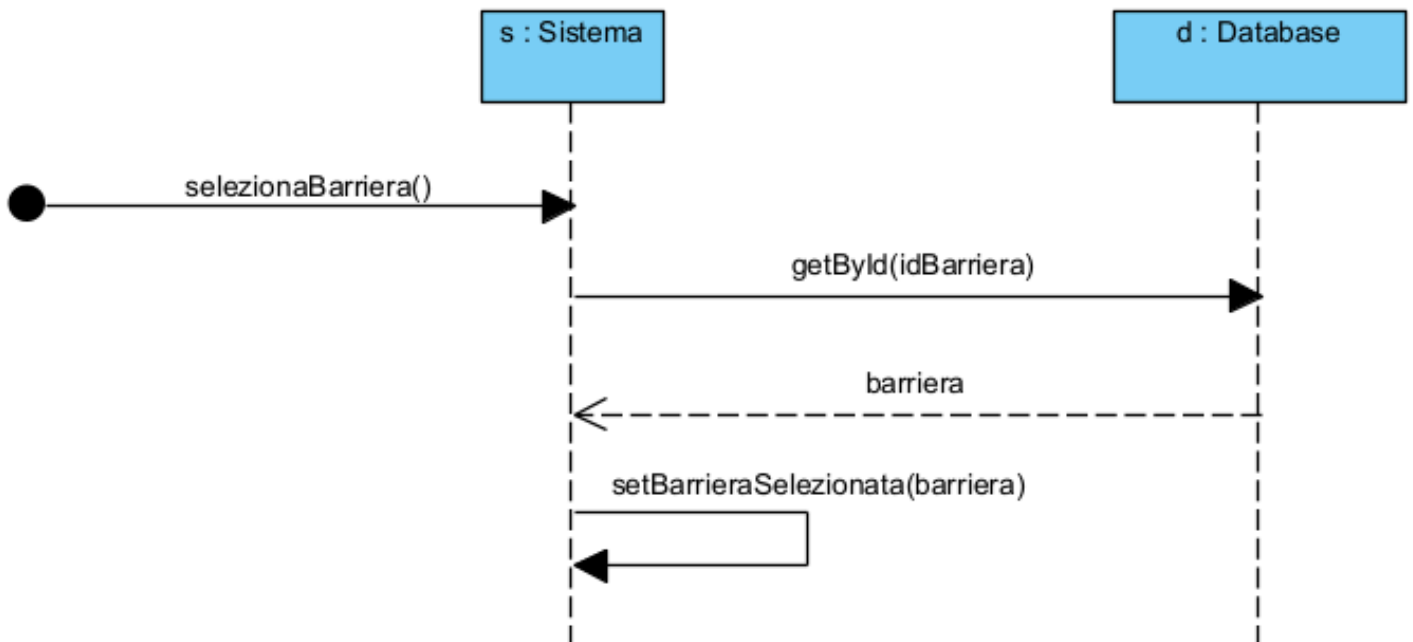
Informazioni



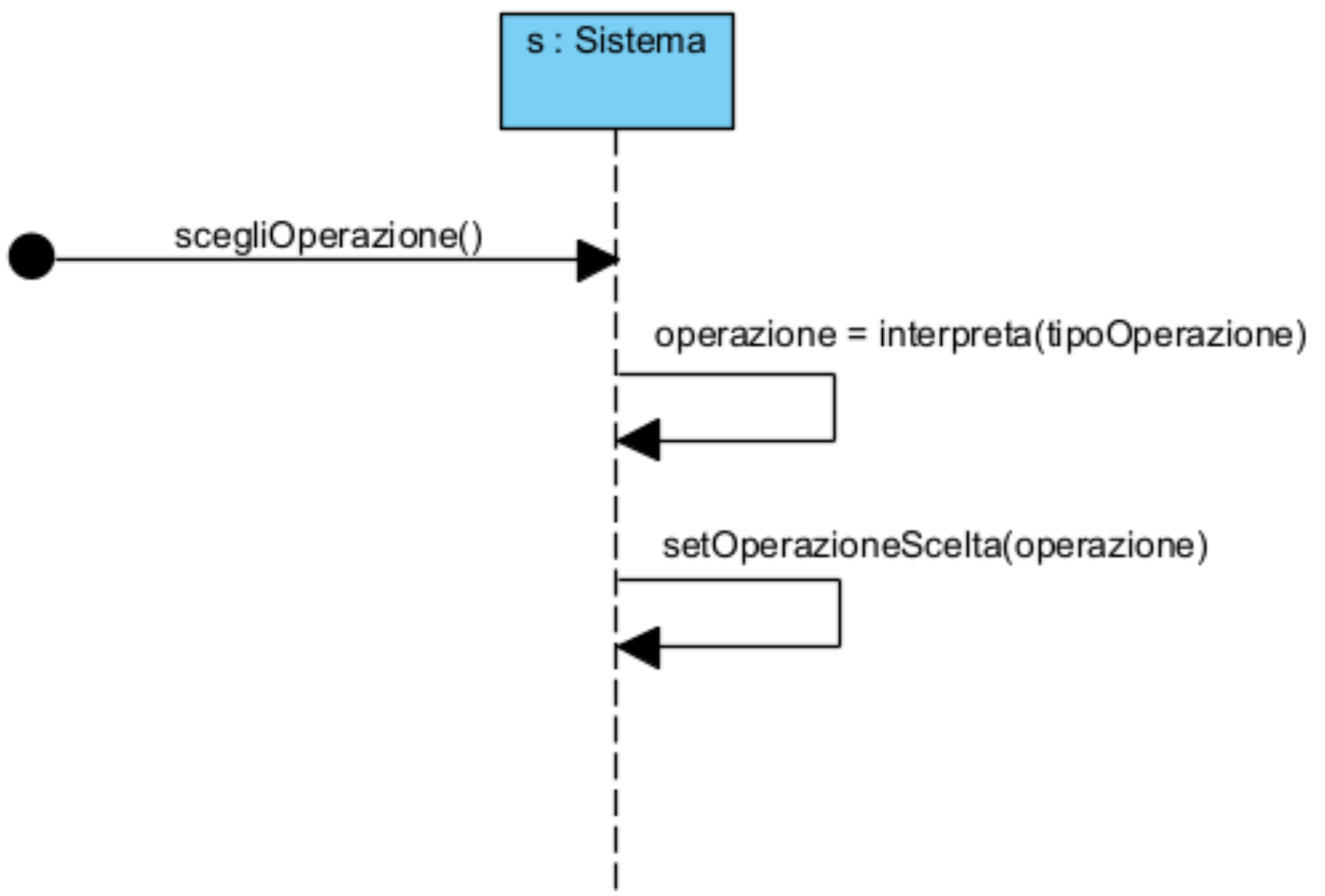
Si noti che **DestinazioneService** e **PercorsoService** non corrispondono a una classe software propria del sistema ma un servizio esterno (*Mapbox Api*, *OpenRouteService Api*). Le chiamate nel diagramma modellano le interazioni logiche con questi servizi.

SEZIONE 3 - Gestione Barriera

- **selezionaBarriera()**

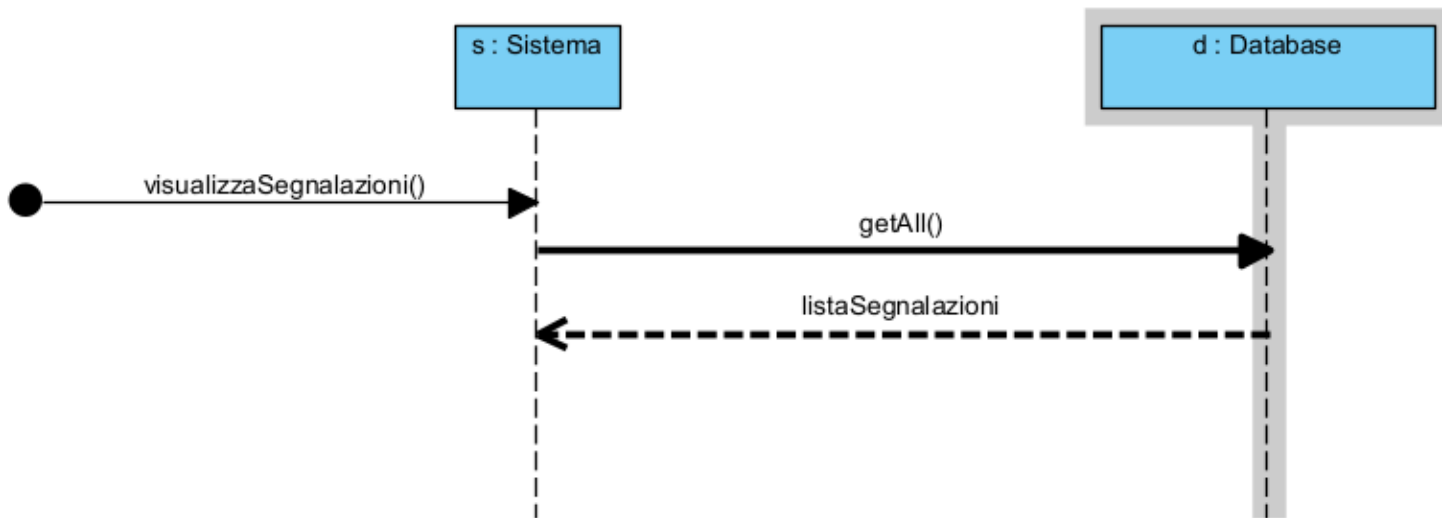


- **scegliOperazione()**

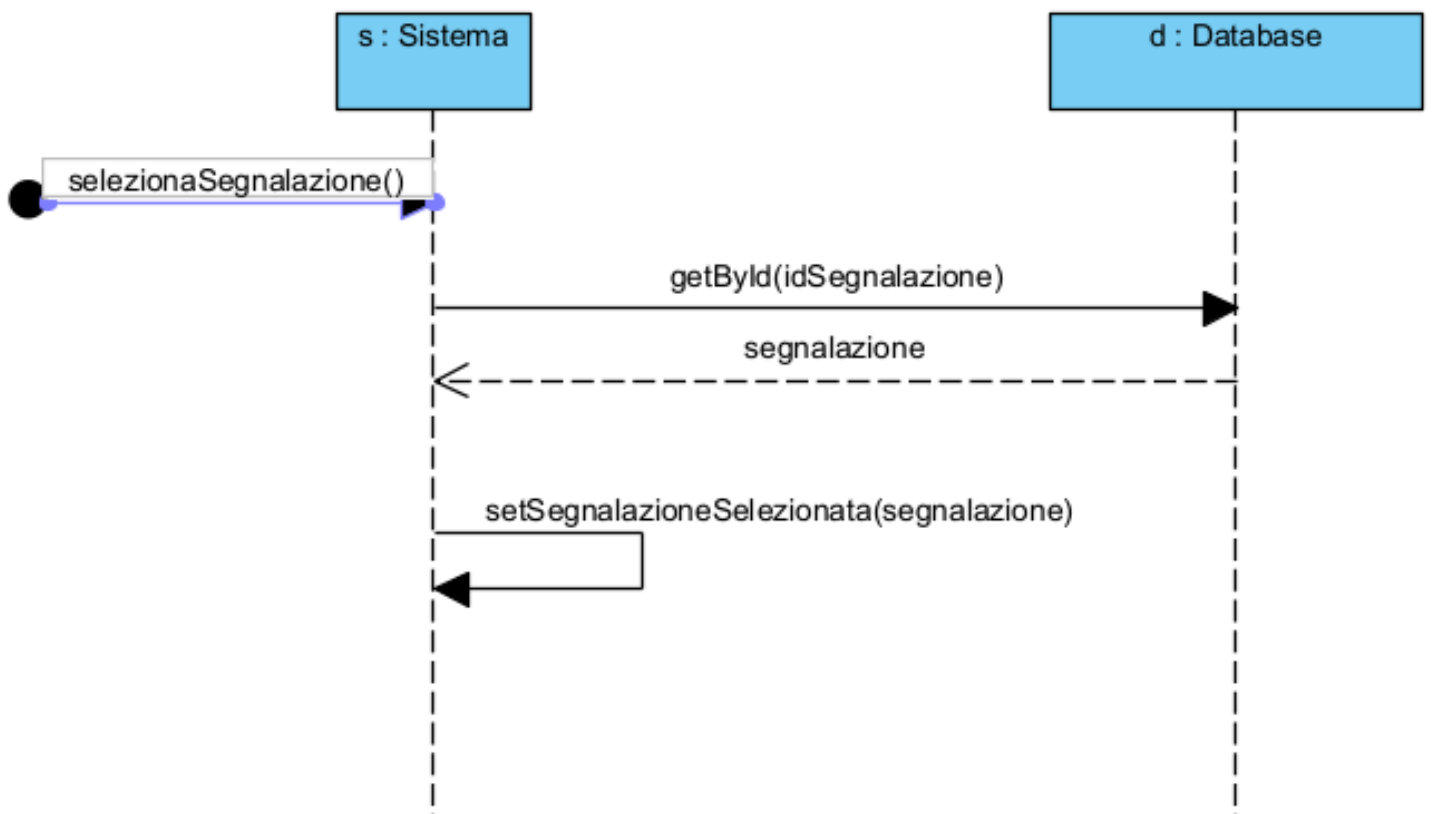


SEZIONE 4 - Gestione Segnalazioni

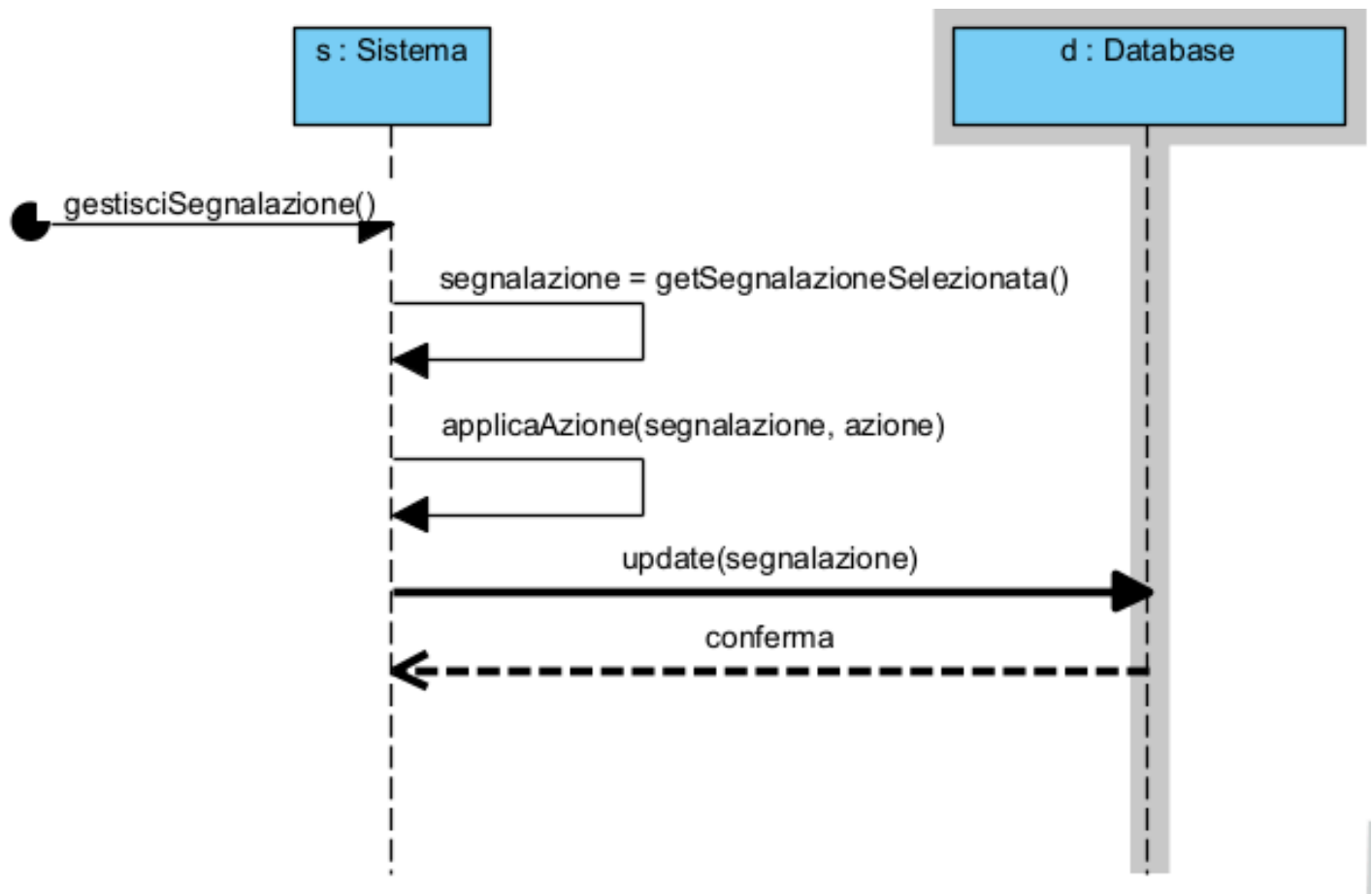
- **visualizzaSegnalazioni()**



- **selezionaSegnalazione()**



- `gestisciSegnalazione()`



Pattern e principi applicati

Pattern GRASP

Sono stati applicati svariati pattern GRASP. Nel dettaglio, i più importanti sono i seguenti:

Componente	Pattern GRASP applicati
metodi loader	● controller per il resto del codice
metodi action	● controller per il resto del codice ● information expert e creator: sono gli unici che creano nuove istanze di oggetti
hook	● high coesion e low copuling: hanno ruoli in uno specifico ambito ● prevengono la creazione di God Components
enum custom	● pure fabrication: non sono rilevati nel modello di dominio, ma sono funzionali da introdurre per semplificare la gestione del codice

Design Pattern

Durante lo sviluppo sono stati applicati diversi design patterns. I più importanti nel dettaglio sono:

- singleton (creazionale): usato ad esempio nel file `db.server.ts`, garantisce che venga creata una sola istanza di connessione al database. Questo previene memory leak e istanze di collegamento al database tramite il client di prisma in stato zombie, ossia non chiuse ma nemmeno utilizzate.
- factory (creazionale): usato ad esempio nel file `icons.ts`. Questo pattern permette di importare solo le icone che servono effettivamente e di recuperarle tramite un identificativo salvato nel database. Questo pattern è stato applicato dopo il tentativo di importare e utilizzare direttamente tutta la libreria `lucide-react`, che però rendeva il caricamento dell'applicazione lento e mandava in crash certe azioni.
- observer (comportamentale): usato nel file `AuthContext.tsx`, dove vengono ascoltati i cambiamenti di stato nell'autenticazione di supabase per rispondere prontamente e triggerare il rendering delle pagine in modo corretto e repentino.

facade (strutturale): usato nei file `supabase.ts` e `firebase.ts`. La complessità delle api di supabase e firebase viene completamente nascosta al resto del codice, vengono esportati solo metodi direttamente utilizzabili all'interno del codice UI.

Pattern Architeturali

L'intera applicazione è stata sviluppata rispettando a pieno gli standard per le applicazioni web react, seguendo vari pattern. Il pattern più importante è MVC. Questo pattern prevede la divisione del codice in tre parti indipendenti e collaboranti:

- **Model:** l'intera logica riguardante il database e la memorizzazione dei dati è gestita dall'ORM Prisma, che astrae la complessità delle query SQL. Questo permette di semplificare notevolmente il codice scritto, in quanto Prisma stesso si occupa di generare una libreria apposita da usare all'interno di codice. Inoltre, la sicurezza è migliorata notevolmente: Prisma è un ORM continuamente testato ed è sicuro da utilizzare.
- **Controller:** questo layer è implementato mediante le funzioni loader e action, presenti ove serve in tutti i file delle rotte. Questi metodi sono esportati e riconosciuti automaticamente da react router come i metodi da chiamare in seguito a richieste GET, POST o di altro tipo. Mediante questi metodi viene validato l'input dell'utente e forniti all'ui i dati necessari.
- **View:** tutti i file delle rotte hanno una funzione esportata di default. Questa funzione si occupa solo ed esclusivamente di mostrare i dati forniti dal loader e le azioni che si possono effettuare.

Design Pattern Martin

Altri design pattern proposti di Martin e applicati all'interno del codice sono i seguenti:

- **Transaction Script:** actions e loaders gestiscono esattamente una richiesta dal layer di presentazione.
- **Data Mapper:** rispettato banalmente per l'utilizzo di Prisma.
- **Page Controller:** il routing permesso da reacto routing è banalmente un page controller che divide l'intera applicazione in più sotto-pagine, ognuna con il suo compito.
- **Identity Field:** ogni oggetto ha associato anche un id univoco (uuid nel nostro caso) per permettere il salvataggio nel database.

Principi SOLID

Sono stati rispettati i principi SOLID. Più in dettaglio:

- **Single Responsibility principle:** ogni hook o componente ha una singola responsabilità.
- **Open/Closed principle:** i componenti sono facilmente estensibili. È sufficiente infatti aggiungere parti di codice, senza che queste rompano il resto del codice già scritto.
- **Liskov Substitution principle:** posso passare ad ogni componente delle properties che estendono quelle che servono. Il componente accetta comunque l'input, considerando solamente le properties a lui necessarie.
- **Interface Segregation principle:** sono stati creati tipi ad hoc da utilizzare nell'ui, invece di passare tipi complessi e con troppi dati ridondanti.
- **Dependency Inversion principle:** i moduli di livello alto non dipendono da quelli di livello basso, ma da loro astrazioni. Un esempio è il fatto che la UI non dipende direttamente da supabase ma da AuthContext.

Qualità del Software

Utilizzo di Git e Github Actions (CI/CD)

Per permettere una collaborazione più agevole tra i vari membri del gruppo è stata utilizzata una repository di Git, in abbinamento con Github. Il branch principale è il main, e la maggior parte delle grandi modifiche o aggiunte di feature è stata effettuata su branch appositi. Finita la modifica è stato effettuato il merge del branch in questione con il main.

Oltre al versionamento, sono stati implementati degli elementi basilari di CI/CD mediante le actions di github. I job svolti in seguito a un push sul main sono nel dettaglio:

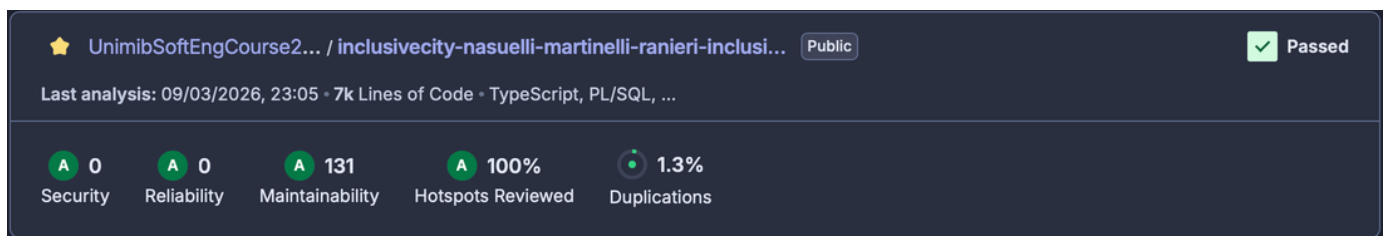
- **test-and-check:** operazione di CI. Questa action si occupa di creare un server Ubuntu temporaneo ed eseguire test di correttezza del codice mediante la generazione del client prisma, il controllo dei tipi di typescript e l'esecuzione dei test scritti con vitest.
- **deploy:** operazione di CD. Al superamento della procedura di CI viene eseguito il deploy del nuovo codice sul branch main sulla piattaforma di hosting vercel. Questo garantisce che la versione in produzione sia effettivamente quella del branch main.
- **analisi di SonarCloud:** configurata direttamente da SonarCloud e non con le actions di github.

Analisi Statica: SonarCloud

La repository è stata controllata, durante lo sviluppo, con il tool SonarCloud, al fine di tenere controllata la presenza di code smell e di tenere alto il livello di pulizia e manutenibilità del codice. Come per le operazioni di CI/CD, ogni push sul branch main scatena una analisi del codice su SonarCloud.

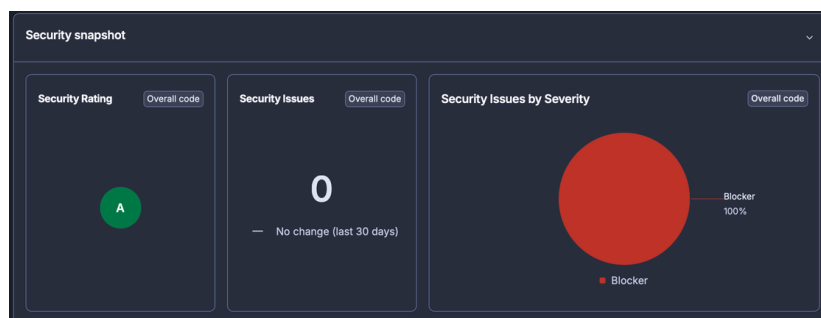
Al fine di velocizzare l'analisi e di considerare solo i file effettivamente utili, è stato inserito un file di configurazione per SonarCloud (sonar-project.properties). In questo file sono stati esclusi file e cartelle da non considerare, come ad esempio la cartella node_modules (delle dipendenze), le cartelle di build e di cache, o le migrazioni di prisma (prisma/migrations): questi sono file generati automaticamente e non devono essere analizzati.

Lo stato corrente del progetto ha un rating di livello A in tutte le categorie, e un livello di duplicazione di codice molto basso, il che indica un ottimo livello di qualità del codice.

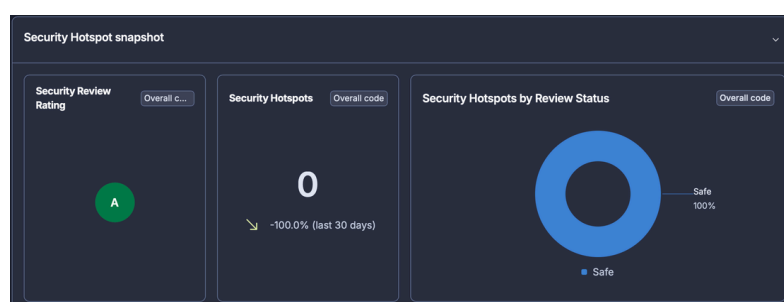


L'analisi ha evidenziato come alcuni problemi fossero dei falsi positivi.

- Security: il grado ottenuto è A. È presente un false positive dovuto al rilevamento di una chiave api hardcoded nel codice. Questa chiave è tuttavia la chiave pubblica del progetto di firebase, necessaria per le notifiche.



- Security Hotspot: anche in questo caso sono risultati dei falsi positivi, dovuti all'utilizzo della funzione random nel file di seed del database. Tutti questi falsi positivi sono stati ricontrollati e segnati come safe, in quanto sono parte di un comportamento voluto.



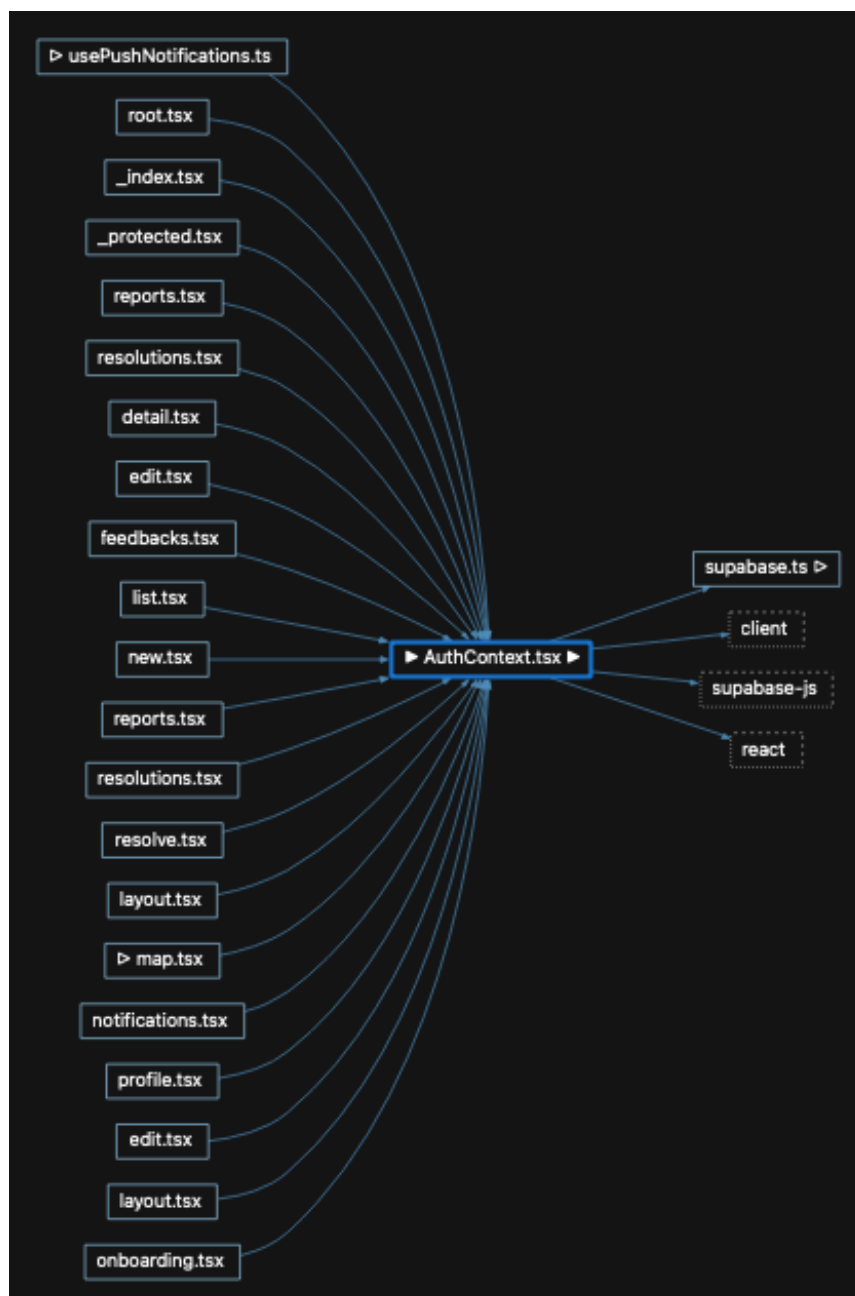
Analisi Antipatterns Strutturali (Report di Understand)

Per validare la robustezza dell'architettura software e individuare potenziali antipattern, il codice sorgente è stato sottoposto a un'analisi qualitativa avanzata utilizzando il tool Understand.

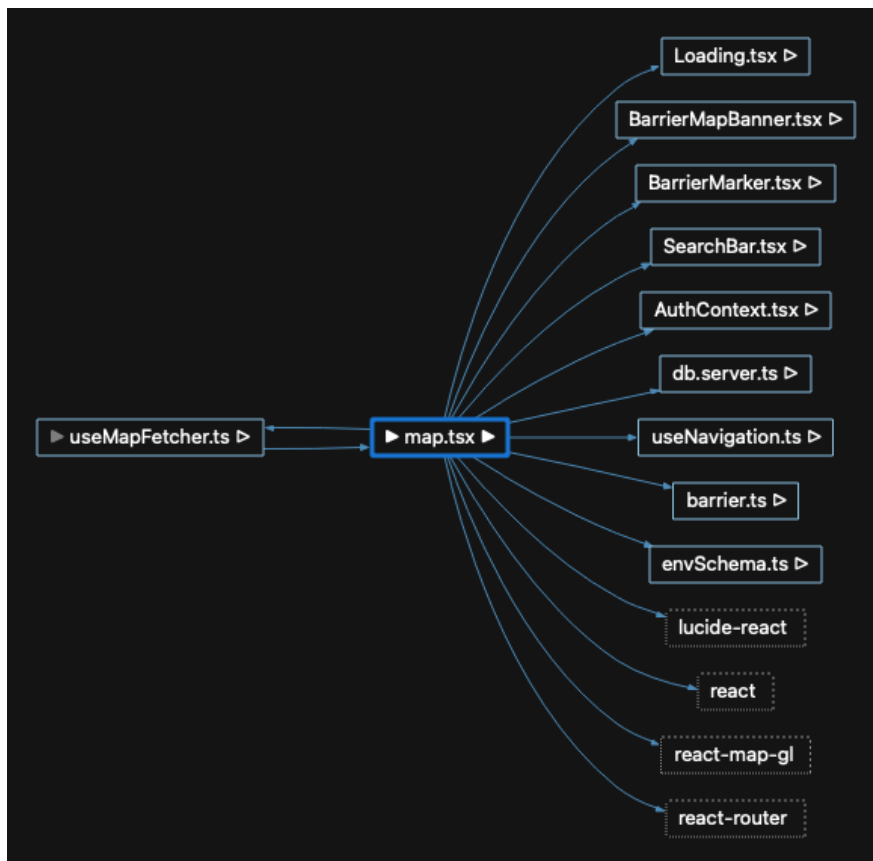
Durante l'analisi sono stati riscontrati problemi con gli alias degli import dovuti ai custom alias di typescript (ossia gli alias degli import che iniziano con ~). Per risolvere il problema è stato necessario creare una versione del progetto in cui tutti gli alias custom sono stati rimpiazzati con import normali.

A seguito dell'analisi dei grafi di dipendenza è stato possibile trovare dei file con ruoli chiave e raggruppare i file per categoria:

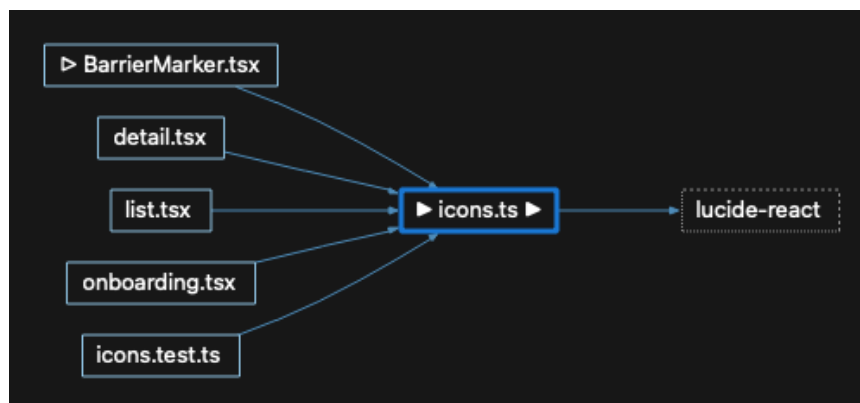
- **Hub:** sono tutti quei file con un alto grado di dipendenze sia in ingresso che in uscita. Un esempio di hub è il file AuthContext.tsx, in quanto è utilizzato da moltissimi altri file e utilizza a sua volta molte altre componenti (sia file che librerie esterne).



- **Breakable:** sono tutti quei file (ad esempio le routes) che hanno un elevatissimo livello di dipendenze. Un esempio è il file `map.tsx`, il quale ha bisogno di dipendenze per autenticazione, accesso al database, mappa e navigazione.



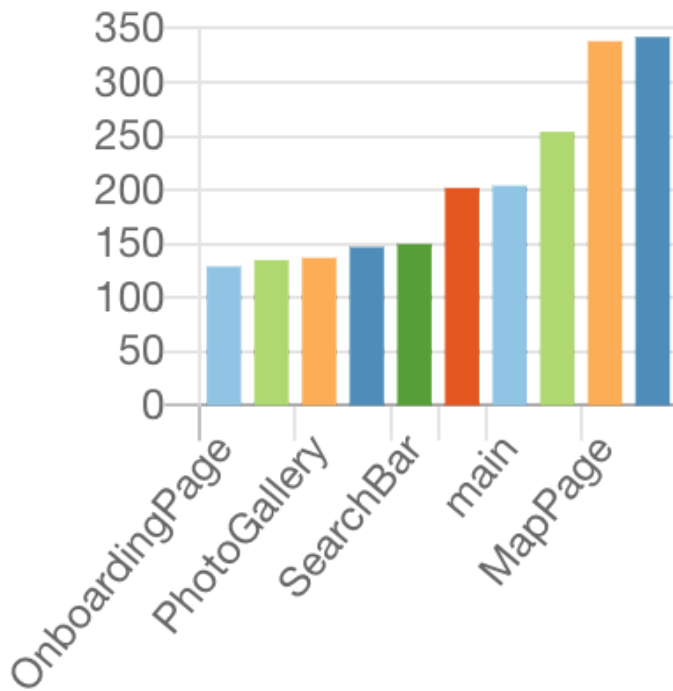
- **Butterfly:** sono tutti quei file che stanno alla base dell'architettura, come ad esempio i file di utils. Sono completamente indipendenti da logiche di business. Un esempio di file di questo tipo è il file `icons.ts`.



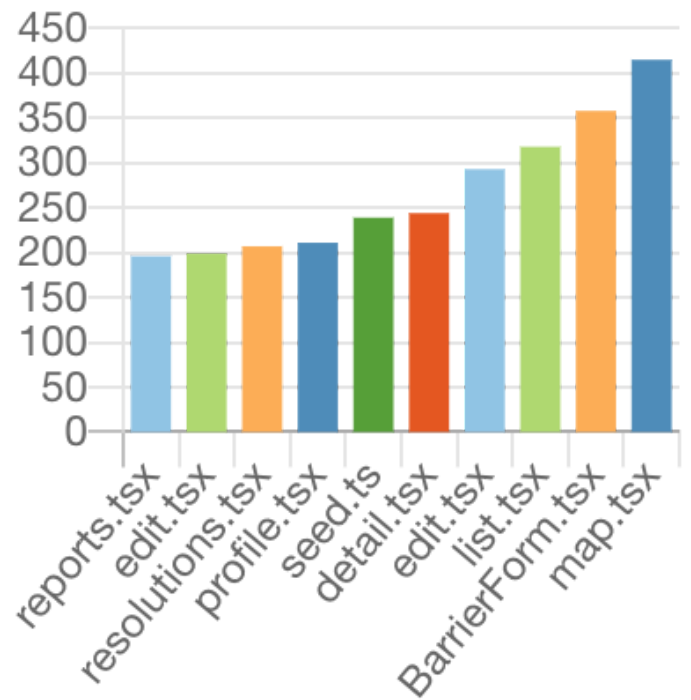


È stata analizzata anche la complessità ciclomatica delle varie funzioni. In tutto il progetto si mantiene a livelli molto bassi, fatta eccezione per due funzioni (BarrierForm e la action di barriers/detail.tsx), dove ha una complessità pari a 10.

Si nota che una funzione con complessità massima 10 è considerata a basso rischio e facile da testare. Il rating della complessità di Understand per questo progetto è A.



Largest functions in file: la funzione più lunga è BarrierForm, con 342 linee di codice.



Largest files: il file più lungo è map.tsx, con 415 linee di codice.

Strategia di Testing

Per validare l'applicazione sono state adottate varie strategie di test:

- test unitari automatizzati: mediante questi test, scritti con vitest, sono state testate le funzioni di utils dell'applicazione. Questi test si occupano ad esempio di validare gli schema di zod utilizzati per la validazione dell'input utente o per le variabili d'ambiente. Sono stati automatizzati mediante le github actions.
- user acceptance tests: a causa della natura complessa e profondamente legata a librerie e servizi esterni (Mapbox gl, navigazione, geolocalizzazione, stato auth di supabase, ...), non è stato possibile testare direttamente l'ui mediante test automatizzati. Si è quindi scelto di validare l'interfaccia grafica mediante test manuali e prove in situazioni reali. Questi test sono stati effettuati con costanza durante tutto lo sviluppo su dispositivi mobili e desktop reali per renderli il più verosimili possibile.

Manuale Utente e Installazione

Link al repository

 GitHub - UnimibSoftEngCourse2022/inclusivecity-nasuelli-martinelli-ranieri-inclusivecity: ev...

Istruzioni per l'installazione

L'applicazione è una pwa, quindi è disponibile via web per mobile e desktop. Può essere utilizzata come sito web oppure come applicazione: per installarla è sufficiente condividere la pagina e salvarla sulla schermata home. Sono state configurate icone e url corretti per gestire entrambi i casi.

L'applicazione è attualmente disponibile al seguente url: [Inclusive City](#).



Avviso

Alcune features dell'applicazione (esempio: navigazione) potrebbero non funzionare correttamente in versione desktop.

Istruzioni dettagliate per l'esecuzione locale

Eseguire una propria copia del progetto è facile e veloce. In dettaglio ecco i passi da eseguire dopo aver clonato la repository da GitHub:

1. installare le dipendenze mediante il comando `npm install`
2. compilare il file `.env` con le chiavi reperite nei seguenti modi (il modello `.env.sample` contiene i nomi esatti da utilizzare):
 - firebase: creare un nuovo progetto e abilitare cloud messaging, dopodichè generare il blocco di chiavi Web Push
 - supabase: creare un nuovo progetto e recuperare url, anon key e stringhe di connessione per il database
 - Mapbox: creare un account gratuito e generare un token pubblico
 - OpenRouteService: creare un account gratuito e copiare la key fornita
3. eseguire il comando `npx prisma migrate reset` per inizializzare il database (nota: verrà generata la libreria di prisma e eseguito anche il file di seed)
4. effettuare il deploy della cloud function su supabase con il comando `npx supabase functions deploy send-notification`
5. creare il webhook del database su supabase per l'invio delle notifiche come illustrato (le altre impostazioni sono quelle di default)

General

Name

send-notification-webhook

Do not use spaces/whitespaces

Conditions to fire webhook

Select which table and events will trigger your webhook

Table

public Notification

This is the table the trigger will watch for changes. You can only select 1 table for a trigger.

Events

☒ Insert
Any insert operation on the table

☐ Update
Any update operation, of any column in the table

☐ Delete
Any deletion of a record

These are the events that are watched by the webhook, only the events selected above will fire the webhook on the table you've selected.

Webhook configuration

Type of webhook

☒ HTTP Request
Send an HTTP request to any URL.

☒ Supabase Edge Functions
Choose a Supabase edge function to run.

Edge Function

Method

POST

Select which edge function to trigger

send-notification

Timeout

5000 ms

Between 1000ms to 10,000ms

6. eseguire il server web con il comando `npm run dev`



Informazioni

Al fine di facilitare il testing, verranno forniti anche un file `.env` già configurato e delle credenziali per accedere ad un account.

Manuale Utente

Scopo dell'applicazione:

Inclusive City è una web app per la visualizzazione e segnalazione di barriere architettoniche presenti sul territorio che mira a rendere più accessibili i luoghi in cui viviamo e ci spostiamo quotidianamente per persone con disabilità di diversi tipi.

Tipologie di Utenti:

- utente non autenticato → potrà accedere solo alla sezione di registrazione/accesso dell'app
- utente autenticato → potrà accedere alla mappa, cercare rotte e visualizzare e gestire le barriere: aggiungendole, segnalandole ed eliminandole. Potrà inoltre gestire il proprio profilo gestendo le notifiche
- admin → potrà gestire i report degli utenti, approvandoli o rifiutandoli e le resolution delle barriere, approvandole e rifiutandole, gestendo in questo modo lo stato delle barriere presenti nell'applicazione insieme alla moderazione automatica dell'applicazione.

Flusso Principale di Utilizzo:

- registrazione / login
- eventuale onboarding con scelta della disabilità
- accesso alla mappa principale
- visualizzazione delle barriere
- ricerca di una destinazione
- visualizzazione del percorso accessibile
- apertura del dettaglio di una barriera
- inserimento / modifica / eliminazione di una barriera
- invio di feedback, report o resolution
- consultazione notifiche
- accesso alla sezione admin se il ruolo è amministratore.

Descrizione delle Schermate:

Per utente non registrato:

- login/registrazione: mostra le opzioni per autenticarsi e/o registrarsi nell'app

Per utente registrato (oltre alle precedenti):

- mappa: mostra barriere nell'area visibile, permette di ricercare/cliccare una destinazione e il percorso per raggiungerla, filtrare le barriere, spostarsi sulla mappa e creare una nuova barriera
- segnala barriera: form per la compilazione di una nuova barriera inserendo un titolo, una descrizione, foto, posizione, tipologia e livello di difficoltà della barriera.
- elenco: mostra l'elenco di tutte le barriere e delle barriere create dall'utente, possibilità di cercarle, filtrarle e crearne di nuove.
- singola barriera: presenta in modo dettagliato i dati della barriera, la foto, l'indirizzo, la descrizione, il livello di difficoltà, la sezione per proporre una risoluzione, la sezione per proporre una valutazione e la sezione per la segnalazione di un problema.
- profilo: mostra i dati del profilo dell'utente, un box per l'attivazione/disattivazione delle notifiche, il centro delle notifiche, la pagina di modifica dell'account, le due sezioni per tornare alla lista delle barriere create e alla lista di tutte le barriere e infine la sezione per fare logout dall'account e tornare alla pagina di registrazione.

Per admin (oltre alle precedenti):

- singola barriera: vengono aggiunte le opzioni di modifica ed eliminazione.
- profilo: viene aggiunta la sezione pannello admin
- pannello admin: propone l'elenco delle segnalazioni dove per ognuna verrà visualizzato il titolo, i dettagli e le opzioni di ignorare il report e di nascondere la barriera. Inoltre propone anche l'elenco delle risoluzioni con, per ognuna la foto relativa e la possibilità di accettare o rifiutare la risoluzione. Infine include anche la possibilità di ricercare le risoluzioni.

Conclusioni

Lo sviluppo di *InclusiveCity* ha rappresentato un'opportunità fondamentale per consolidare e mettere in pratica i concetti teorici appresi e per utilizzare a pieno gli strumenti incontrati nel corso.

Dal punto di vista tecnico, il progetto ci ha permesso di esplorare un moderno stack full-stack, affrontando sfide reali legate all'integrazione di API cartografiche (Mapbox, OpenRouteService), alla gestione di infrastrutture Cloud (supabase e firebase) e all'implementazione di sistemi guidati da eventi.

Oltre a questo, il progetto ci ha permesso di costruire un progetto completo in tutte le sue parti, e di realizzare elaborati nelle fasi di analisi e progettazione che hanno portato alla scrittura di codice pulito e con una idea chiara di ciò che era da fare.

L'applicazione dei principi di design del software e di assegnazione delle responsabilità ci ha portato a costruire, pezzo per pezzo, un software finale funzionante.

Il continuo controllo della qualità mediante le analisi di SonarCloud e Understand e l'integrazione di una pipeline di CI/CD con dei test eseguiti automaticamente sono stati di grande aiuto per tenere alta la qualità del codice stesso.